

Ferialpraktikum Sommer 2026

Maximilian Daxböck

July 2026

1 Wahrscheinlichkeiten allgemein

In der Wahrscheinlichkeitsrechnung geht es um die Beschreibung von Zufallsexperimenten, also um Versuche mit unklarem Ausgang.

1.1 Allgemeines

Jedes Zufallsexperiment hat eine Menge Ω ('Grundraum' bzw. 'sample space'), die alle möglichen Versuchsausgänge $\omega \in \Omega$ ('Elementarereignisse' bzw. 'elementary event') beinhaltet. Ein Ereignis ('Event') E , das eintreten kann, ist eine Teilmenge $E \subseteq \Omega$.

Ist ein Ereignis A gegeben, so kann man diesem eine Wahrscheinlichkeit $P(A)$ zuordnen. Dabei misst P , wie wahrscheinlich A eintritt. Die Funktion P muss allerdings nicht für alle Teilmengen von Ω , sondern nur für eine Auswahl $\mathcal{A}$ (' σ -Algebra') davon definiert sein. Außerdem müssen dabei folgende Axiome nach Kolmogorov erfüllt sein:

1. Positiv für alle möglichen Ereignisse: $\forall A \subseteq \mathcal{A} : P(A) \geq 0$
2. Normiert: $P(\Omega) = 1$
3. Aus $E_1 \cap E_2 = \{\}$ folgt für beliebige Ereignisse E_1 und E_2 mit dieser Eigenschaft immer, dass $P(E_1 \cup E_2) = P(E_1) + P(E_2)$.

Das Tripel $(\Omega, \mathcal{A}, P)$ wird dann Wahrscheinlichkeitsraum ('probability space') genannt.

1.2 Zufallsvariablen

Eine reellwertige Zufallsvariable ('random variable') ist eine Funktion $X : \Omega \rightarrow \mathbb{R}$, die Elementen des Grundraums eine Zahl zuordnet. Die Wahrscheinlichkeit, dass X in einem Intervall $I \subset \mathbb{R}$ liegt, kann dann durch

$$P(X \in I) = P(\{\omega \in \Omega : X(\omega) \in I\})$$

bestimmt werden. Allerdings muss die Menge $(X \in I)$ in $\mathcal{A}$ liegen.

Eine Funktion P_X heißt Verteilung ('distribution'), wenn sie direkt einer Menge B die Wahrscheinlichkeit $P_X(B) = P(X \in B)$ zuordnet. Es gibt verschiedene Arten solcher Verteilungen:

1. Diskret: Eine Wahrscheinlichkeitsverteilung ist 'diskret', wenn $\exists T \subset \mathbb{R}$ mit $|T| \leq |\mathbb{N}|$, sodass

$$P_X(x) = \begin{cases} P(X = x), & \text{falls } x \in T \\ 0 & \text{sonst} \end{cases}.$$

Eine Funktion $f_X(x) = P_X(x)$ wird in diesem Kontext 'probability mass function' genannt.

2. Stetig: Eine Wahrscheinlichkeitsverteilung heißt stetig ('continuous'), wenn für sie eine Dichte ('density') f_X existiert, sodass

$$P_X(A) = \int_A f_X(x) dx.$$

Die Funktion $F(x) = P_X((-\infty, x))$ wird auch als Verteilung ('cumulative distribution function') bezeichnet.

1.3 Zusammenfassende Statistik

Der Erwartungswert $E[x]$ einer Zufallsvariablen X ist im diskreten Fall als

$$E[x] = \sum_{x \in \mathbb{R}} xP(X = x)$$

und im stetigen Fall als

$$E[x] = \int_{\mathbb{R}} xf_X(x)dx$$

definiert. Er misst den durchschnittlichen Wert, den X annimmt. Die Varianz ist die durchschnittliche quadratische Abweichung vom Mittel- bzw. Erwartungswert und wird durch

$$\sigma^2 = \text{Var}[x] = E[(x - E[x])^2]$$

gebildet.

Es gilt weiter

$$\text{Var}[x] = E[(x - E[x])^2] = E[x^2] - E[x]^2.$$

Beweis. Im Falle, dass X diskret ist:

$$\begin{aligned} E[x] &= \sum_x (x - E[x])^2 P(X = x) \\ &= \sum_x x^2 P(X = x) - \sum_x 2xE[x]P(X = x) + \sum_x E[x]^2 P(X = x) \\ &= E[x^2] - 2E[x] \sum_x xP(X = x) + E[x]^2 \sum_x P(X = x) \\ &= E[x^2] - 2E[x]^2 + E[x]^2 \\ &= E[x^2] - E[x]^2 \end{aligned}$$

Wenn X stetig ist:

$$\begin{aligned} E[(x - E[x])^2] &= \int_{\mathbb{R}} (x - E[x])^2 f_X(x)dx \\ &= \int_{\mathbb{R}} x^2 f_X(x)dx - \int_{\mathbb{R}} 2E[x]xf_X(x)dx + \int_{\mathbb{R}} E[x]^2 f_X(x)dx \\ &= E[x^2] - 2E[x] \int_{\mathbb{R}} xf_X(x)dx + E[x]^2 \int_{\mathbb{R}} f_X(x)dx \\ &= E[x^2] - 2E[x]^2 + E[x]^2 \\ &= E[x^2] - E[x]^2 \end{aligned}$$

Das stimmt also sowohl für diskrete als auch für stetige Zufallsvariablen. $\square$

1.4 Bedingte Wahrscheinlichkeiten, Unabhängigkeit und Satz von Bayes

Die bedingte Wahrscheinlichkeit, dass ein Ereignis A eintritt, unter der Annahme, dass B bereits eingetreten ist, wird durch

$$P(A \cap B) = P(A|B)P(B)$$

berechnet. Zwei Ereignisse A und B heißen zudem genau dann 'unabhängig', wenn $P(A \cap B) = P(A|B)P(B)$, bzw. $P(A|B) = P(A)$. Dieser Zusammenhang gilt auch für Wahrscheinlichkeitsverteilungen und -dichten.

Der Satz von Bayes

$$P(H|E) = \frac{P(E|H)P(H)}{P(E)}$$

kann einfach durch Gleichsetzen der gemeinsamen Verteilung $P(E \cap H) = P(E|H)P(H) = P(H|E)P(E)$ hergeleitet werden. Er stellt einen Zusammenhang zwischen den Wahrscheinlichkeiten einer angenommenen Hypothese H und einem Beleg E für diese Hypothese her. Hier heißt $P(H)$ 'prior', da er eine vorherige Annahme über die Wahrscheinlichkeit von H widerspiegelt. $P(E)$ ist der 'evidence', also die Wahrscheinlichkeit, die beobachteten Daten E wirklich zu beobachten. Zudem ist $P(E|H)$ die Likelihood der Hypothese. Sie gibt an, wie wahrscheinlich es ist, E unter der Hypothese H zu beobachten. Der Ausdruck $P(H|E)$ beschreibt die 'posterior'-Wahrscheinlichkeit der Hypothese H , also die veränderte Wahrscheinlichkeit von H , nachdem E beobachtet wurde.

2 Frequentistische vs. Bayes'sche Statistik

Angenommen, ein Bernoulli-Experiment, also ein Zufallsexperiment mit zwei möglichen Ausgängen und konstanter Erfolgswahrscheinlichkeit wird n mal durchgeführt, wobei es k mal positiv und $n-k$ Mal negativ ausgeht. Die Erfolgswahrscheinlichkeit θ ist dabei allerdings unbekannt. Ziel ist es nun, eine gute Schätzung θ abzugeben. Hier kommen nun zwei verschiedene Auffassungen von Statistik ins Spiel.

2.1 Frequentistische Vorgehensweise

Der frequentistische Ansatz θ zu schätzen, wäre es, das Verhältnis der positiven Ergebnisse zu allen Versuchen zu bestimmen, also

$$\hat{\theta}_{MLE} = \frac{k}{n}.$$

Diese Vorgehensweise kann durch die Maximum-Likelihood-Methode folgendermaßen gerechtfertigt werden: Die Wahrscheinlichkeit bei gegebenem θ k positive Beobachtungen zu machen, folgt einer Binomialverteilung (Die Zufallsvariable X beschreibt die Anzahl der positiven Fälle):

$$P(X = k|\theta) = L(\theta|X = k) = \binom{n}{k} \theta^k (1 - \theta)^{n-k}$$

Möchte man nun eine gute Schätzung für θ finden, so muss man diese Likelihood maximieren. Daher:

$$\frac{dL}{d\theta} = 0$$

$$\frac{d}{d\theta} \left(\binom{n}{k} \theta^k (1 - \theta)^{n-k} \right) = 0$$

$$k\theta^{k-1}(1 - \theta)^{n-k} - (n - k)\theta^k(1 - \theta)^{n-k-1} = 0$$

$$k(1 - \theta) - (n - k)\theta = 0$$

$$k - n\theta = 0$$

$$\hat{\theta}_{MLE} = \frac{k}{n}$$

Somit wäre $\hat{\theta}_{MLE}$ der ideale Parameterwert... oder doch nicht?

2.2 Bayes'scher Ansatz

Der zweite Ansatz, dieses Problem der Parameterbestimmung zu lösen, ist der Bayes'sche. Hier geht man davon aus, dass man durch das Experiment nicht die ganze Information über θ erhält. Stattdessen bezieht man noch eine vorherige Annahme (Prior) über die Verteilung der Werte von θ mit ein, wodurch man eine neue Wahrscheinlichkeitsverteilung für θ erhält. Mit der Likelihood (von vorhin) und diesem Prior kann man nun nach dem Satz von Bayes die Posterior-Verteilung von θ nach den Beobachtungen bestimmen. Dadurch kann man auch mit jedem weiteren Experiment die Wahrscheinlichkeitsverteilung anpassen und erhält ein immer genaueres Modell.

Der Vorteil im Vergleich zur frequentistischen Vorgehensweise ist, dass man nicht nur einen einzelnen Wert für θ bekommt, sondern die ganze Verteilung von θ schätzt. Somit weiß man, mit welcher Sicherheit ein Parameter zu einem guten Modell führt. Braucht man tatsächlich einen Wert für θ , dann kann man einfach die Posterior-Verteilung maximieren (MAP, Maximum-a-posteriori). Der Nachteil eines Bayes'schen Modells ist, dass es wesentlich komplizierter ist, als die MLE und man einen gut passenden Prior finden muss.

In dem Fall des vorherigen Zufallsexperiments wäre die Beta-Verteilung ein guter Prior für θ . Die Beta-Verteilung mit Parametern α und β ist definiert als

$$p(\theta) = \frac{1}{B(\alpha, \beta)} \theta^{\alpha-1} (1 - \theta)^{\beta-1},$$

wobei $B(\alpha, \beta) = \frac{\Gamma(\alpha)\Gamma(\beta)}{\Gamma(\alpha+\beta)}$ der Normierung dient und $\theta \in [0; 1]$.

Mit diesem Prior erhält man für die Posterior-Verteilung des Parameters θ durch den Satz von Bayes:

$$p(\theta|X = k) = \frac{p(X = k|\theta)p(\theta)}{p(X = k)}.$$

Weil der Nenner nicht von θ abhängt, also konstant ist, kann er als Proportionalitätsfaktor weggelassen werden. Um ihn auszurechnen, müsste das Integral

$$p(X = k) = \int_0^1 p(X = k|\theta)p(\theta)d\theta$$

berechnet werden, was unnötig kompliziert ist. Daher berechnen wir

$$p(\theta|X = k) \propto p(X = k|\theta)p(\theta)$$

stattdessen und normieren nachher. Also:

$$p(X = k|\theta)p(\theta) = \binom{n}{k} \theta^k (1 - \theta)^{n-k} \cdot \frac{1}{B(\alpha, \beta)} \theta^{\alpha-1} (1 - \theta)^{\beta-1} \propto \theta^{k+\alpha-1} (1 - \theta)^{n-k+\beta-1}$$

Das zeigt, dass

$$p(\theta|X = k) \propto \theta^{k+\alpha-1} (1 - \theta)^{n-k+\beta-1}$$

wieder eine Beta-Verteilung sein muss. Es gilt dann mit richtiger Normierung:

$$p(\theta|X = k) = \frac{1}{B(k + \alpha, n - k + \beta)} \theta^{k+\alpha-1} (1 - \theta)^{n-k+\beta-1}$$

Das Maximum dieser Posterior-Verteilung $\hat{\theta}_{MAP}$ wird ähnlich berechnet, wie bei der Maximum-Likelihood-Schätzung und ist

$$\hat{\theta}_{MAP} = \frac{k + \alpha - 1}{n + \alpha + \beta - 2}.$$

Das zeigt jetzt mehrere Dinge: Einerseits sehen wir, dass ein Beta-Prior, wenn die Likelihood einer Binomialverteilung folgt, auch zu einem Beta-Posterior mit angepassten α und β führt. Das bedeutet, dass man ein solches 'Update' von α und β nochmals durchführen könnte, wobei wieder eine Beta-Verteilung herauskommen würde. Da die Binomialverteilungs-Likelihood und der Beta-Prior immer zu einem Beta-Posterior führen, ist die Beta-Verteilung der konjugierte Prior der Binomialverteilung. Andererseits finden wir dadurch auch eine Interpretation für die ursprünglichen Parameter α und β . In diesem Kontext kann $\alpha - 1$ als die hypothetische, als Prior angenommene Anzahl positiver Versuchsausgänge gelesen werden. $\beta - 1$ ist die Anzahl der hypothetischen negativen Versuchsausgänge. Außerdem zeigt sich, dass sich die MAP-Schätzung für große n der MLE annähert.

2.3 Beispiel

Als Beispiel habe ich hier ein kleines Experiment durchgeführt: Ich werfe ein Tixo 20 Mal auf den Tisch und messe, wie oft das Tixo wenn es aufkommt stehen bleibt oder liegt. Ich nehme davor (also als Prior) an, dass das Tixo ca. doppelt so oft liegt wie es stehen bleibt. Dann führe ich ein Experiment mit 20 Versuchen durch. Dabei bleibt das Tixo 2 Mal stehen. Die Maximum-Likelihood-Schätzung wäre also

$$\hat{\theta}_{MLE} = \frac{2}{20} = 0.1$$

Nun der Bayes'sche Ansatz: Für den Prior nehme ich an eine Beta-Verteilung mit $\alpha = 5$ und $\beta = 9$ an. Der Posterior ist also auch wieder eine Beta-Verteilung mit $\alpha_{neu} = \alpha + k = 7$ und $\beta_{neu} = \beta + n - k = 27$. Das Maximum der Posterior-Verteilung liegt bei

$$\hat{\theta}_{MAP} = \frac{6}{32} = 0.1875$$

Die Prior- und Posterior-Verteilung ist in Abbildung 1 zu sehen. Wenn ich jetzt noch ein Experiment durchführe (30 Versuche, 3 Mal stehend, 27 Mal liegend), kann ich ganz einfach die Posterior Wahrscheinlichkeit von vorhin anpassen und erhalte eine Beta-Verteilung mit $\alpha = 10$ und $\beta = 54$. Der Vergleich mit dem vorherigen Posterior (der für diese Verteilung der Prior war) und dem ursprünglichen Prior ergibt den Graphen in Abbildung 2. Man sieht, dass der Modus der Verteilung von θ (also der MAP-Estimate) sich immer weiter einem guten Wert annähert und, dass sich die Verteilung immer weiter enger um diesen Wert zentriert.



Abbildung 1: Ergebnisse des Experiments: Prior in grün, Posterior in rot, MLE als schwarze Linie

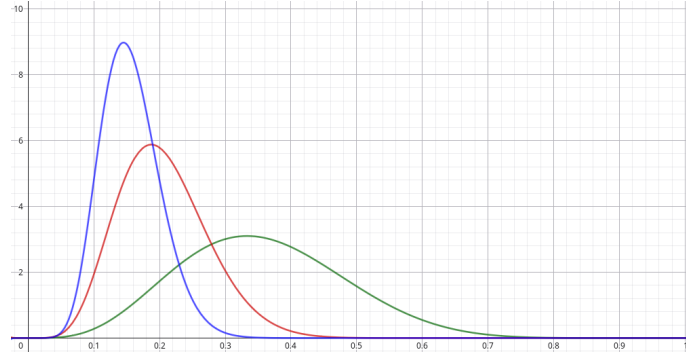


Abbildung 2: Verteilungen im Vergleich: vorheriger Prior in grün und vorheriger Posterior in rot, an die zweiten Daten angepasste Posterior-Verteilung in blau

2.4 Diskreter Naive Bayes-Classifier

Folgende Problemstellung: Es gibt N Inputs x_j , $j = 1, \dots, N$, die aus jeweils D Features bestehen, welche eine endliche Anzahl F an Werten annehmen können. Jeder Input $x_j = (x_{j,1}, \dots, x_{j,D})$ besitzt zudem ein Label y_j , das für eine von K möglichen Klassen steht; jeder Input gehört somit einer bestimmten Klasse an. Das Ziel ist es jetzt, ein Modell zu finden, das neue Inputs basierend auf den gegebenen Daten, einer passenden Klasse zuordnen kann. Für diese Zuordnung wollen wir immer die Klasse benutzen, für die die Wahrscheinlichkeit am größten ist, also

$$\text{vorhergesagte-Klasse}(x) = \underset{y \in \{1, \dots, K\}}{\operatorname{argmax}} P(Y = y | X = x).$$

Die Zufallsvariable Y steht hier für die Klasse und $X = (X_1, \dots, X_D)$ steht für die Inputs. Die Posterior-Wahrscheinlichkeit der Klasse $P(Y = y | X = x)$ ist nach dem Satz von Bayes proportional zum Produkt von Likelihood und Prior, das heißt

$$P(Y = y | X = x) \propto P(X = x | Y = y) P(Y = y).$$

Zudem nehmen wir auf naive Weise (daher der Name) an, dass die einzelnen Features des Inputs unabhängig verteilt sind. Durch diese Annahme kann man die Likelihood als Produkt der bedingten Wahrscheinlichkeiten der einzelnen Features umschreiben:

$$P(Y = y | X = x) \propto P(Y = y) \prod_{d=1}^D P(X_d = x_d | Y = y).$$

Um diese Wahrscheinlichkeiten anzunähern, nutzen wir die relativen Häufigkeiten. Somit ergibt sich für die Prior-Wahrscheinlichkeiten der einzelnen Klassen k :

$$P(Y = k) = \frac{|\{j = 1, \dots, N : y_j = k\}|}{N}$$

Die bedingten Wahrscheinlichkeiten der Features werden durch

$$P(X_d = x_d | Y = k) = \frac{|\{j = 1, \dots, N : x_{j,d} = x_d \wedge y_j = k\}|}{|\{j = 1, \dots, N : y_j = k\}|}$$

bestimmt.

Damit die Wahrscheinlichkeiten für die einzelnen Klassen bei einem gegebenen Input (bis auf eine Proportionalitätskonstante) in einem funktionierenden Modell effizient berechnet werden können, müssen die gerade beschriebenen Wahrscheinlichkeiten alle einmal ausgerechnet und in einer 'Lookup'-Tabelle festgehalten werden. Diese besteht aus einer Liste $p \in [0; 1]^K$, $p_k = P(Y = k)$ mit den Prior-Wahrscheinlichkeiten der einzelnen Klassen und einem Tensor $L \in [0; 1]^{K \times D \times F}$ vom Rang 3, der die Likelihoods $L_{k,d,f} = P(X_d = f | Y = k)$ beinhaltet.

Um schließlich eine Vorhersage für die Klasse eines gegebenen Inputs $x = (x_1, \dots, x_D)$ zu machen, muss man dann nur noch

$$\text{vorhergesagte-Klasse}(x) = \underset{k \in \{1, \dots, K\}}{\operatorname{argmax}} p_k \prod_{d=1}^D L_{k,d,x_d}$$

berechnen.

Es stellt sich heraus, dass zu einem binären Naive Bayes-Classifer (also mit nur zwei Klassen 0 und 1) ein analoges Lineares Klassifikationsmodell existiert. Das heißt,

$$\exists b \in \mathbb{R}, w = (w_1, \dots, w_D)^T \in \mathbb{R}^D, \phi : \mathbb{R}^D \rightarrow \mathbb{R}^D : \underset{y \in \{0,1\}}{\operatorname{argmax}} P(Y = y | X = x) = 1 \iff w^T \phi(x) + b > 0.$$

Beweis. Aus

$$\underset{y \in \{0,1\}}{\operatorname{argmax}} P(Y = y | X = x) = 1$$

folgt, dass $P(Y = 1 | X = x) > P(Y = 0 | X = x)$ bzw. weil Wahrscheinlichkeiten immer positiv sind

$$\begin{aligned} \frac{P(Y = 1 | X = x)}{P(Y = 0 | X = x)} &= \frac{P(X = x | Y = 1)P(Y = 1)}{P(X = x | Y = 0)P(Y = 0)} \\ &= \frac{P(Y = 1)}{P(Y = 0)} \prod_{d=1}^D \frac{P(X_d = x_d | Y = 1)}{P(X_d = x_d | Y = 0)} > 1 \end{aligned}$$

Setzt man $a_d := P(X_d = 1 | Y = 1)$ und $b_d := P(X_d = 1 | Y = 0)$, dann gilt

$$P(X_d = x_d | Y = 1) = a_d^{x_d} (1 - a_d)^{1-x_d}$$

und

$$P(X_d = x_d | Y = 0) = b_d^{x_d} (1 - b_d)^{1-x_d}.$$

Das kann einfach überprüft werden, indem man in beide Gleichungen die beiden möglichen Werte für x_k (d.h. 0 in 1) einsetzt, wodurch sich wahre Aussagen ergeben. Diese Zusammenhänge kann man nun zusammen mit $p := P(Y = 1)$ in die Ungleichung von vorhin einsetzen, wodurch man

$$\frac{p}{1-p} \prod_{d=1}^D \frac{a_d^{x_d} (1 - a_d)^{1-x_d}}{b_d^{x_d} (1 - b_d)^{1-x_d}} > 1$$

erhält. Logarithmieren und Umformen führt zu

$$\log\left(\frac{p}{1-p} \prod_{d=1}^D \frac{a_d^{x_d} (1 - b_d)^{x_d-1}}{b_d^{x_d} (1 - a_d)^{x_d-1}}\right) > 0.$$

Durch Gruppieren der Terme mit und ohne x_d erhält man

$$\log\left(\frac{p}{1-p} \prod_{d=1}^D \frac{1 - a_d}{1 - b_d}\right) + \log\left(\prod_{d=1}^D \left(\frac{a_d(1 - b_d)}{b_d(1 - a_d)}\right)^{x_d}\right) > 0$$

und weiter

$$\log\left(\frac{p}{1-p} \prod_{d=1}^D \frac{1-a_d}{1-b_d}\right) + \sum_{d=1}^D x_d \log\left(\frac{a_d(1-b_d)}{b_d(1-a_d)}\right) > 0.$$

Setzt man jetzt noch $b := \log\left(\frac{p}{1-p} \prod_{d=1}^D \frac{1-a_d}{1-b_d}\right)$ und $w = (w_1, \dots, w_D)$ mit $w_d = \log\left(\frac{a_d(1-b_d)}{b_d(1-a_d)}\right)$, dann ergibt sich

$$b + \sum_{d=1}^D w_d x_d > 0$$

bzw.

$$w^T x + b > 0.$$

Das ist genau die Gleichung für ein lineares Klassifizierungsmodell mit $\phi(x) = x$. Somit existiert also ein solches lineares Modell mit b, w und ϕ , das dem Bayes'schen Modell gleich ist. $\square$

2.5 Anwendung: Email-Spam-Filter

Ich habe diese Theorie angewandt, um einen Email-Spam-Filter zu programmieren. Dazu habe ich folgende Annahmen bzw. Entscheidungen getroffen:

1. Es gibt genau zwei Klassen ($K = 2$), denen die E-Mails zugeordnet werden können: Spam (1) oder Ham (=nicht Spam, 0)
2. E-Mails werden als 200-dimensionale Vektoren ($D = 200$) dargestellt, wobei jeder Eintrag ein Feature ist, das eine Charakteristik des E-Mails beschreibt. Alle Features können außerdem nur die Werte 0 oder 1 annehmen ($F = 2$). Wie genau dieses Embedding funktioniert weiß ich nicht, aber es ist bereits gegeben (Also die Emails liegen bereits als Vektoren vor).
3. Die Daten stammen aus dem Enron-Datensatz und sind in Test- und Trainingsdaten unterteilt. Der Trainingsdatensatz besteht aus 26972 gelabelten E-Mails, die Testdaten haben 6744 solcher Einträge, wobei in beiden Datensätzen ungefähr gleich viele Spam- wie Ham-E-mails vorhanden sind.

Das beschriebene Modell habe ich in Python implementiert (d.h. p und T berechnen lassen und Vorhersagen treffen) und es funktioniert einigermaßen gut. Die Confusion-Matrix der Testdaten lautet

$$C_1 = \begin{bmatrix} 2711 & 598 \\ 208 & 3227 \end{bmatrix};$$

das Modell schätzt also die meisten E-Mails (ca. 88.05%) richtig ein. Leider werden aber auch ca. 18.07% der Nicht-Spam-E-mails als Spam gewertet. Einem guten Spam-Filter sollte das besser nicht passieren.

2.6 Anderer Ansatz und Vergleich

Ich habe dann noch einen zweiten (nicht Bayes'schen) Spam-Filter zum Vergleich programmiert. Dazu habe ich ein binäres Klassifikationsmodell

$$f(x) = \sigma(c^T \cdot \text{ReLU}(Ax + b))$$

mit $A \in \mathbb{R}^{200 \times 2}$, $b, c \in \mathbb{R}^2$ und $\sigma(z) = 1/(1 + \exp(-z))$ erstellt. Der Output $f(\cdot)$ nimmt reelle Werte zwischen 0 und 1 an, wobei 0 für Ham und 1 für Spam steht. Je nachdem, wo ich den Threshold setze, ab dem auf- oder abgerundet wird (ein Hyperparameter), kann ich das Modell noch weiter anpassen.

Die richtigen Werte finde ich ganz einfach (ohne Bayes), indem ich die Likelihood der Parameter bezüglich der Daten maximiere bzw. die negative Log-Likelihood

$$L(A, b, c) = - \sum_{k=1}^{26972} [y_k \log f(x_k | A, b, c) + (1 - y_k) \log(1 - f(x_k | A, b, c))]$$

minimiere. Das mache ich mithilfe von Gradient Descent mittels PyTorch. Das Ergebnis (mit Rundungs-Treshold 0.6, das ist meiner durch Probieren entstandenen Meinung nach der beste Wert) sieht als Confusion-Matrix

$$C_2 = \begin{bmatrix} 3129 & 180 \\ 175 & 3259 \end{bmatrix}$$

besser aus, als jenes des Naiven Bayes Classifier.

Allerdings darf man, wenn man die beiden Modelle vergleicht, nicht nur die Confusion-Matrix betrachten, sondern muss auch den Rechenaufwand miteinbeziehen. Denn obwohl das zweite Modell zwar ein besseres Ergebnis liefert, hat es einige Minuten gedauert, bis es fertig war. Das Naive Bayes-Modell hingegen performierte zwar minimal schlechter, war aber auch in ein paar Sekunden fertig und einsatzbereit - Schließlich mussten ja nur p und L einmal bestimmt werden und keine 25000 Gradient Descent Schritte wie für A , b und c waren nötig. Somit haben beide Modelle Vor- und Nachteile.

3 Sampling-Methoden

Um ein Zufallswerte zu finden, die nach gegebenen Wahrscheinlichkeitsverteilungen verteilt sind, verwendet man gewisse Sampling-Methoden. Hier sind ein paar Beispiele dafür.

3.1 Categorical Sampling

Das Categorical Sampling ist eine Sampling-Methode, die es - unter der Voraussetzung, dass man von $\mathcal{U}(0,1)$ (also der Gleichverteilung auf dem Intervall von 0 bis 1) sampeln kann - von jeder beliebigen diskreten Wahrscheinlichkeitsverteilung Samples bilden kann.

Angenommen, es gibt eine diskrete Zufallsvariable X , die die Werte $x_1, x_2, \dots, x_n$ für ein $n \in \mathbb{N}$ mit den jeweiligen Wahrscheinlichkeiten $\mu_1, \mu_2, \dots, \mu_n \geq 0$ annimmt. Wie kann man Werte von X sampeln?

Die Lösung: Die Summe aller Wahrscheinlichkeiten ist per Definition $\sum_{j=1}^n \mu_j = 1$. Deshalb ist es möglich, Intervalle $I_k \subset (0; 1]$ mit

$$I_k := \left(\sum_{j=1}^{k-1} \mu_j; \sum_{j=1}^k \mu_j \right]$$

so zu bestimmen, dass für ein $u \sim \mathcal{U}(0,1)$ für alle solchen Intervalle

$$P(u \in I_k) = \int_{I_k} \mathcal{U}(0,1) du = \int_{I_k} du = \sum_{j=1}^k \mu_j - \sum_{j=1}^{k-1} \mu_j = \mu_k.$$

Das heißt, um Werte für X zu sampeln, muss man nur ein $u \sim \mathcal{U}(0,1)$ sampeln und bestimmen, in welchem Intervall I_k die Zahl u liegt. Dann ist das entsprechende x_k das Sample der Zufallsvariable X .

Ich habe diesen eben beschriebenen Algorithmus des Categorical Sampling mithilfe von Python implementiert. Das Ergebnis für die beispielhafte Verteilung mit $P(X=0) = \mu_1 = 0.5$, $P(X=1) = \mu_2 = 0.1$, $P(X=2) = \mu_3 = 0.3$ und $P(X=3) = \mu_4 = 0.1$ mit 1000 Samples ist in Abbildung 3 veranschaulicht.

Auch habe ich dieses Sampling-Tool angewandt, um Markov-Ketten zu simulieren. Das funktioniert einfach, indem man eine Menge an Anfangs-Samples hat, für jedes dieser Samples, die Wahrscheinlichkeitsverteilung für den nächsten Schritt aus der Übergangsmatrix abzulesen und jeweils ein Sample von dieser Verteilung zu generieren. Dann werden die alten Samples durch die neuen ersetzt und der Prozess wiederholt sich. Somit kann man Markov-Ketten einfach simulieren.

3.2 Gibbs Sampling

Das Gibbs Sampling ist eine Sampling-Methode, die es ermöglicht, von einer von mehreren Variablen abhängigen stetigen Wahrscheinlichkeitsverteilung zu sampeln. Angenommen, die gemeinsame Dichte $f(x,y)$ zweier Zufallsvariablen ist zu kompliziert, als dass man davon Samples bilden könnte. Dann kann man stattdessen das Gibbs-Sampling verwenden.

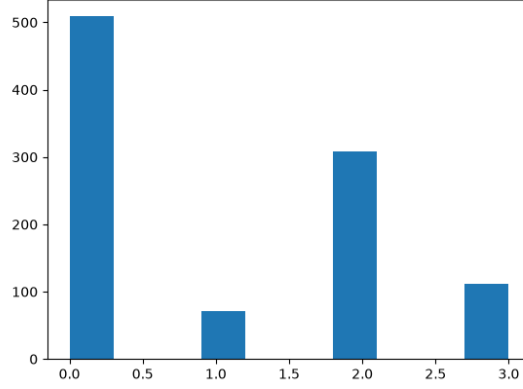


Abbildung 3: Categorical Sampling Implementation Showcase

Beim Gibbs-Sampling nimmt man zunächst einen beliebigen Wert x_0 für eine der Zufallsvariablen x an. Dann entnimmt man ein Sample y_1 von $f(y|x = x_0)$ und berechnet anschließend ein Sample x_1 von $f(x|y = y_1)$. Das erhaltene Paar (x_1, y_1) wird abgespeichert und kann als Sample verwendet werden. Durch wiederholen dieses Schritts (wobei allerdings x_1 als ursprünglicher Wert angenommen wird usw.) entstehen immer mehr Samples. Schließlich erhält man eine lange Liste (bzw. ein Markov-Kette) von Sample-Paaren, von der man jedoch die ersten k Paare verwirft (weil x_0 beliebig war und die Verteilung der Samples verändern könnte). k nennt man 'Burn-in'. Die finale Liste enthält dann die gewünschten Samples.

4 Bayes'sche Lineare Regression

Bei der Bayes'schen Linearen Regression geht es darum, aus $\beta_0 \in \mathbb{R}$ und $\beta_1 \in \mathbb{R}^d$ bestehende lineare Modelle

$$y_k = \beta_0 + \beta_1^T x_k + \varepsilon_k$$

zu finden, die die Daten $(x_k, y_k) \in \mathbb{R}^d \times \mathbb{R}$, $1 \leq k \leq n$ mit $\varepsilon_k \sim \mathcal{N}(0, \sigma^2)$ beschreiben. Die einzelnen ε_k sind i.i.d. (also unabhängig voneinander und nach der gleichen Verteilung verteilt). Es geht dabei allerdings nicht darum, ideale Werte für β_0 und β_1 zu bestimmen, sondern darum, die Verteilung dieser Parameter zu finden. Möchte man mit dem Modell Vorhersagen treffen, muss man von der Verteilung Samples bilden und kann diese (oder den Durchschnitt) als Modelle verwenden. Somit erhält man nicht nur ein gutes Modell, sondern weiß auch, wie sicher das Modell richtig liegt.

Das Ziel ist es jetzt also, die Likelihood von β (und auch σ^2) zu finden, einen geeigneten Prior für die beiden zu bestimmen, und den Posterior auszurechnen. Dann wollen wir von der Posterior-Verteilung Samples bilden, die dann als Modell genutzt werden können.

4.1 Finden der Posterior-Verteilung der Parameter

Zunächst verändern wir das Modell etwas, damit alles ein bisschen übersichtlicher wird und keine vielen Indizes gebraucht werden. Wir definieren also $\beta = (\beta_0, \beta_1^T)^T \in \mathbb{R}^m$, $m = d + 1$, $y = (y_1, \dots, y_n)^T \in \mathbb{R}^n$, $\varepsilon = (\varepsilon_1, \dots, \varepsilon_n)^T \in \mathbb{R}^n$ und

$$X = \begin{bmatrix} 1 & x_1^T \\ \vdots & \vdots \\ 1 & x_n^T \end{bmatrix} \in \mathbb{R}^{n \times m},$$

wobei X auch als 'Design-Matrix' bezeichnet wird. Das Modell lautet nun also für alle Datenpunkte parallel

$$y = X\beta + \varepsilon.$$

Da $\varepsilon \sim \mathcal{N}(0, \sigma^2 I)$ einer n -dimensionalen Normalverteilung folgt, muss y ebenso verteilt sein - eben nur um $X\beta$ verschoben:

$$y \sim \mathcal{N}(X\beta, \sigma^2 I)$$

Somit gilt für die Dichte von y (bzw. die Likelihood von β und σ^2):

$$\begin{aligned} P(y, X|\beta, \sigma^2) &= L(\beta, \sigma^2|y, X) = \frac{1}{\sqrt{(2\pi)^n \det(\sigma^2 I)}} \exp\left(-\frac{1}{2}(y - X\beta)^T (\sigma^2 I)^{-1} (y - X\beta)\right) \\ &= \frac{1}{(\sqrt{2\pi}\sigma)^n} \exp\left(-\frac{1}{2\sigma^2}(y - X\beta)^T (y - X\beta)\right) \\ &= \frac{1}{(\sqrt{2\pi}\sigma)^n} \exp\left(-\frac{1}{2\sigma^2}\|y - X\beta\|_2^2\right). \end{aligned}$$

Nehmen wir jetzt einen aus zwei Teilen bestehenden 'hierarchischen' Prior für β und σ^2 an:

$$f(\beta, \sigma^2) = f(\beta|\sigma^2)f(\sigma^2)$$

Dabei treffen wir folgende Entscheidungen:

1. $f(\beta|\sigma^2)$ folgt einer Normalverteilung mit Erwartungswert μ_0 und Covarianz-Matrix $\sigma^2 \Sigma_0$.
2. $f(\sigma^2)$ ist eine Inverse-Gamma Verteilung mit den Parametern a_0 und b_0 . Die Dichte von σ^2 (d.h. NICHT von σ ohne Quadrat) ist also:

$$f(\sigma^2) = \frac{b_0^{a_0}}{\Gamma(a_0)} \left(\frac{1}{\sigma^2}\right)^{a_0+1} \exp\left(-\frac{b_0}{\sigma^2}\right)$$

Jetzt wollen wir die Posterior-Dichte von β und σ^2 (d.h. $f(\beta, \sigma^2|y, X)$) berechnen:

$$\begin{aligned} f(\beta, \sigma^2|y, X) &\propto L(\beta, \sigma^2|y, X) f(\beta|\sigma^2) f(\sigma^2) \\ &\propto \frac{1}{\sigma^n} \exp\left(-\frac{1}{2\sigma^2}(y - X\beta)^T (y - X\beta)\right) \frac{1}{\sigma^m} \exp\left(-\frac{1}{2\sigma^2}(\beta - \mu_0)^T \Sigma_0^{-1} (\beta - \mu_0)\right) \frac{1}{\sigma^{2a_0+2}} \exp\left(-\frac{b_0}{\sigma^2}\right) \\ &= \frac{1}{\sigma^{2a_0+m+n+2}} \exp\left(-\frac{1}{2\sigma^2}[(y - X\beta)^T (y - X\beta) + (\beta - \mu_0)^T \Sigma_0^{-1} (\beta - \mu_0) + 2b_0]\right). \end{aligned}$$

Dieser Ausdruck ist übrigens proportional zu $f(\sigma^2|\beta, y, X)$ und wird später noch interessant werden. Jetzt aber weiter zur Posterior-Berechnung. Damit wir den Posterior $f(\beta, \sigma^2|y, X)$ (auch proportional zur vorherigen Gleichung) wieder in $f(\beta, \sigma^2|y, X)$ und $f(\sigma^2|y, X)$ aufteilen können, müssen wir im Exponenten quadratisch ergänzen. Dazu betrachten wir erstmal nur den Exponenten:

$$(y - X\beta)^T (y - X\beta) + (\beta - \mu_0)^T \Sigma_0^{-1} (\beta - \mu_0) + 2b_0$$

(ausmultiplizieren)

$$= y^T y - \beta^T X^T y - y^T X \beta + \beta^T X^T X \beta + \beta^T \Sigma_0^{-1} \beta - \beta^T \Sigma_0^{-1} \mu_0 - \mu_0^T \Sigma_0^{-1} \beta + \mu_0^T \Sigma_0^{-1} \mu_0 + 2b_0$$

(gruppieren der in β quadratischen und linearen Terme)

$$= \beta^T (X^T X + \Sigma_0^{-1}) \beta - [\beta^T (X^T y + \Sigma_0^{-1} \mu_0) + (y^T X + \mu_0^T \Sigma_0^{-1}) \beta] + y^T y + \mu_0^T \Sigma_0^{-1} \mu_0 + 2b_0$$

(Substitution: $\Lambda := X^T X + \Sigma_0^{-1}$)

$$= \beta^T \Lambda \beta - [\beta^T (X^T y + \Sigma_0^{-1} \mu_0) + (y^T X + \mu_0^T \Sigma_0^{-1}) \beta] + y^T y + \mu_0^T \Sigma_0^{-1} \mu_0 + 2b_0$$

Diesen Term wollen wir jetzt quadratisch ergänzen. Das heißt, wir suchen erst einmal ein α , sodass sich $(\beta - \alpha)^T \Lambda (\beta - \alpha)$ nur durch eine Konstante von dem oberen Term unterscheidet. Multipliziert man diese Quadratische Form aus, erhält man:

$$(\beta - \alpha)^T \Lambda (\beta - \alpha) = \beta^T \Lambda \beta - \beta^T \Lambda \alpha - \alpha^T \Lambda \beta + \alpha^T \Lambda \alpha$$

Für α muss also gelten:

$$\beta^T (X^T y + \Sigma_0^{-1} \mu_0) = \beta^T \Lambda \alpha$$

bzw.

$$\alpha = \Lambda^{-1} (X^T y + \Sigma_0^{-1} \mu_0)$$

Jetzt können wir den obigen Term quadratisch ergänzen. Es ergibt sich

$$(\beta - \alpha)^T \Lambda (\beta - \alpha) - \alpha^T \Lambda \alpha + y^T y + \mu_0 \Sigma_0^{-1} \mu_0 + 2b_0.$$

Das kann man jetzt wieder in den Exponenten einsetzen, wodurch wir

$$f(\beta, \sigma^2 | y, X) \propto \frac{1}{\sigma^{2a_0+m+n+2}} \exp\left(-\frac{1}{2\sigma^2}[(\beta - \alpha)^T \Lambda (\beta - \alpha) - \alpha^T \Lambda \alpha + y^T y + \mu_0 \Sigma_0^{-1} \mu_0 + 2b_0]\right)$$

erhalten. Trennen wir nun den in Bezug auf β quadratischen Teil vom von β unabhängigen, stellt sich heraus, dass β wieder normalverteilt ist und σ^2 wieder einer Inverse-Gamma-Verteilung folgt:

$$\begin{aligned} &= \frac{1}{\sigma^m} \exp\left(-\frac{1}{2\sigma^2}(\beta - \alpha)^T \Lambda (\beta - \alpha)\right) \cdot \frac{1}{(\sigma^2)^{\frac{n}{2}+a_0+1}} \exp\left(-\frac{1}{2\sigma^2}(2b_0 - \alpha^T \Lambda \alpha + y^T y + \mu_0 \Sigma_0^{-1} \mu_0)\right) \\ &\propto f(\beta | \sigma^2, y, X) f(\sigma^2 | y, X) \end{aligned}$$

Somit ist $\beta | \sigma^2, y, X \sim \mathcal{N}(\alpha, \sigma^2 \Lambda^{-1})$ und $\sigma^2 | y, X \sim \text{Inv-Gamma}(a_0 + \frac{n}{2}, b_0 + \frac{1}{2}(y^T y + \mu_0 \Sigma_0^{-1} \mu_0 - \alpha^T \Lambda \alpha))$. Der Prior und der Posterior von β und von σ^2 sind beide von der 'gleichen Art'. Es stellt sich heraus, dass die Normalverteilung sowie die Inverse Gamma-Verteilung konjugierte Priors zur (normalverteilten) Likelihood sind.

4.2 Sampling und Implementation

Nachdem wir jetzt die Posterior-Verteilung der Parameter kennen, möchten wir davon Samples bekommen, um konkrete Samples für mögliche Modelle zu erhalten. Dafür nutzen wir das Gibbs-Sampling. Wir brauchen also $f(\beta | \sigma^2, y, X)$ und $f(\sigma^2 | \beta, y, X)$. Den ersten Ausdruck kennen wir bereits, denn $\beta | \sigma^2, y, X \sim \mathcal{N}(\alpha, \sigma^2 \Lambda^{-1})$. Die zweite Verteilung ist allerdings noch unbekannt. Dafür nutzen wir ein Zwischenergebnis aus der vorherigen Rechnung. Daraus folgt

$$f(\sigma^2 | \beta, y, X) \propto \frac{1}{\sigma^{2a_0+m+n+2}} \exp\left(-\frac{1}{2\sigma^2}[(y - X\beta)^T (y - X\beta) + (\beta - \mu_0)^T \Sigma_0^{-1} (\beta - \mu_0) + 2b_0]\right)$$

bzw.

$$\sigma^2 | \beta, y, X \sim \text{Inverse-Gamma}(a_0 + \frac{m+n}{2}, b_0 + \frac{1}{2}[(y - X\beta)^T (y - X\beta) + (\beta - \mu_0)^T \Sigma_0^{-1} (\beta - \mu_0)]).$$

Nachdem beide bedingten Wahrscheinlichkeiten jetzt durch eine bekannte Verteilung verteilt sind, kann man einfach das Gibbs-Sampling einsetzen und erhält Samples für β (und auch σ^2 , aber die sind weniger interessant).

Ich habe diese theoretischen Rechnungen in ein Code-Template eingebaut, das für $m = 2$, $\sigma^2 \Sigma_0 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$, $\mu_0 = (0, 0)^T$, $a_0 = b_0 = 1$ für jeweils 0, 2, 5 und 20 Datenpunkte $(x, y) \in \mathbb{R}^2$ die Dichte von $\beta | \sigma$ anzeigt und zugehörige Samples (β_0, β_1) als Geraden $y = \beta_0 + \beta_1 x$ visualisiert. Das Resultat ist in Abbildung 4 zu sehen.

Man erkennt eindeutig, dass je größer n ist (also je mehr Datenpunkte), desto enger liegen die gesampelten Geraden beieinander. Allerdings erkennt man das nicht nur an den Plots der Geraden, sondern auch an den Posterior-Verteilungen von β_0 und β_1 : Je mehr Daten, desto geringer ist die Streuung in der Dichte.

5 Lagrange Multiplikatoren

Lagrange-Multiplikatoren bieten eine Methode, um restringierte Optimierungsprobleme - das sind Optimierungsprobleme mit Nebenbedingungen - zu lösen.

5.1 Die Idee dahinter

Gegeben sei eine Funktion $f : \mathbb{R}^n \rightarrow \mathbb{R}$, die optimiert werden soll. Zudem gibt es eine weitere Funktion $g : \mathbb{R}^n \rightarrow \mathbb{R}$. f soll jetzt nicht auf ganz $\mathbb{R}^n$ optimiert werden, sondern nur Punkte x mit $g(x) = 0$ sind als Lösung des Optimierungsproblems zugelassen. Wie kann man ein solches Optimum finden?

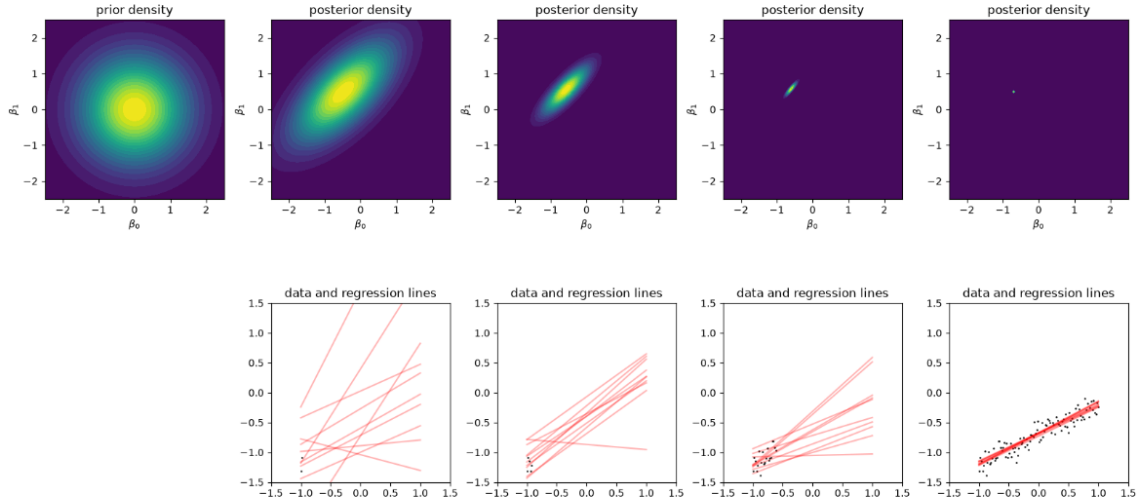


Abbildung 4: Bayesian Linear Regression mit $n = 0$ (entspricht dem Prior), 2, 5, 20, 100 Datenpunkten, Veranschaulichung der Dichte der Parameter und einiger Samples

Einen strukturierten Ansatz liefern die Lagrange-Multiplikatoren: Im Bereich $g(x) = 0$ muss der Gradient $\nabla g(x)$ an jeder Stelle x orthogonal auf den Bereich (z.B. eine Kurve) stehen. Das liegt daran, dass der Gradient in die Richtung des steilsten Anstieges zeigt und somit auf die 'Höhenlinie' $g(x) = 0$ normal stehen muss. Sucht man jetzt ein Optimum x^* von f in dem angesprochenen Bereich, dann darf sich $f(x)$ entlang $g(x) = 0$ um $f(x^*)$ nicht verändern. Das bedeutet, dass der Gradient $\nabla f(x^*)$ ebenfalls orthogonal zum Bereich $g(x) = 0$, und somit parallel zu $\nabla g(x^*)$, sein muss. Das heißt, für ein Optimum x^* der Funktion $f(x)$ unter der Einschränkung $g(x)$ muss gelten:

$$\nabla f(x^*) = \lambda \nabla g(x^*), \lambda \in \mathbb{R}$$

und

$$g(x^*) = 0$$

Diese beiden Bedingungen kann man folgendermaßen mithilfe der sog. Lagrange-Funktion $L(x, \lambda) = f(x) + \lambda g(x)$ zusammenfassen:

$$\nabla_{x,\lambda} L(x, \lambda) = 0$$

Denn

$$\frac{\partial L}{\partial x} = 0 \implies \nabla f(x) + \lambda \nabla g(x) \quad (\text{Bedingung 1})$$

und

$$\frac{\partial L}{\partial \lambda} = 0 \implies g(x) = 0 \quad (\text{Bedingung 2}).$$

Somit wird aus dem restringierten Optimierungsproblem ein nicht-restringiertes Optimierungsproblem. Womöglich, wird es dadurch komplexer, denn es gibt eine neue Variable λ , die auch beachtet werden muss, aber man muss nicht mit einer Restriktion arbeiten, die man womöglich Parametrisieren oder kompliziert umformen muss. Außerdem könnte man (meiner Einschätzung nach) einfach mit Gradient-Descent dadurch ein kompliziertes Problem schneller lösen.

5.2 Beispiel

Folgendes Rechenbeispiel: Gegeben sei die Funktion $f(x, y) = 2 - x^2 - 2y^2$. Ich möchte ein Maximum finden, das auf der Gerade $x + y = 1$ liegt (Nebenbedingung).

Zuerst löse ich das Beispiel ohne Lagrange-Multiplikatoren:

1. Nebenbedingung umformen: $y = 1 - x$
2. In f einsetzen: $f(x, 1 - x) = 2 - x^2 - 2(1 - x)^2 = -3x^2 + 4x$

3. Maximum (x) ausrechnen: $\frac{df}{dx} = 0 \implies -6x + 4 = 0 \implies x = \frac{2}{3}$

4. y berechnen: $y = 1 - x = 1 - \frac{2}{3} = \frac{1}{3}$

Das Maximum liegt also bei

$$x = \frac{2}{3}, y = \frac{1}{3}.$$

Dasselbe Problem kann man auch über Lagrange-Multiplikatoren lösen:

1. Lagrange-Funktion aufstellen: $L(x, y, \lambda) = f(x, y) + \lambda g(x, y) = 2 - x^2 - 2y^2 + \lambda(x + y - 1)$

2. Gradient bestimmen und Null setzen:

$$\nabla_{x,y,\lambda} L = \begin{bmatrix} \partial L / \partial x \\ \partial L / \partial y \\ \partial L / \partial \lambda \end{bmatrix} = \begin{bmatrix} -2x + \lambda \\ -4y + \lambda \\ x + y - 1 \end{bmatrix} = 0$$

Dadurch entsteht das Gleichungssystem

$$\begin{bmatrix} -2 & 0 & 1 \\ 0 & -4 & 1 \\ 1 & 1 & 0 \end{bmatrix} \begin{bmatrix} x \\ y \\ \lambda \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}.$$

3. Gleichungssystem mittels Gauß-Verfahren lösen ergibt:

$$x = \frac{2}{3}, y = \frac{1}{3}, \lambda = \frac{4}{3}.$$

Es zeigt sich, dass die Technik mit den Lagrange-Multipliern dasselbe Optimum liefert, wie das 'händische' Lösen durch Umformen und Einsetzen.

6 Principal Component Analysis (PCA)

Die PCA ist eine Methode mit der man aus hochdimensionalen Daten die Hauptkomponenten, die die wesentliche Information der Daten beinhalten, bestimmen kann. Das ermöglicht es, die Dimension der Daten zu reduzieren (bzw. die Daten auf den von den wichtigsten Hauptkomponenten aufgespannten Unterraum zu projizieren), wodurch Kompression, Datenvisualisierung usw. einfach möglich werden. Zuvor aber ein paar benötigte Grundlagen...

6.1 Orthogonalität der Eigenvektoren symmetrischer Matrizen

Gegeben sei eine symmetrische Matrix A (d.h. $A^T = A$). Dann stehen alle Eigenvektoren (mit verschiedenen Eigenwerten) jeweils orthogonal zueinander.

Beweis. Seien v_1 und v_2 zwei verschiedene Eigenvektoren von A mit voneinander verschiedenen Eigenwerten λ_1 und λ_2 . D.h.

$$Av_1 = \lambda_1 v_1$$

und

$$Av_2 = \lambda_2 v_2$$

mit $v_1 \neq v_2$ und $\lambda_1 \neq \lambda_2$. Dann gilt für den Ausdruck $v_1^T Av_2$ wegen der Symmetrie von A :

$$v_1^T Av_2 = v_1^T A^T v_2 = (Av_1)^T v_2 = \lambda_1 v_1^T v_2$$

und

$$v_1^T Av_2 = v_1^T \lambda_2 v_2 = \lambda_2 v_1^T v_2.$$

Damit:

$$\implies \lambda_1 v_1^T v_2 = \lambda_2 v_1^T v_2$$

bzw.

$$(\lambda_1 - \lambda_2) v_1^T v_2 = 0.$$

Weil $\lambda_1 \neq \lambda_2$ muss zwangsläufig

$$v_1^T v_2 = 0 \iff v_1 \perp v_2.$$

□

6.2 Orthogonale Projektionen auf lineare Unterräume

Im Fall, dass wir einen beliebigen Vektor x auf einen Vektor u mit $\|u\|_2 = 1$ projizieren möchten, so ist die Koordinate y , die alleine ausreicht, um die Projektion von x auf den von u aufgespannten Unterraum zu beschreiben, gegeben durch

$$y = u^T x.$$

Möchten wir den projizierten Punkt jetzt als Punkt x_{proj} auf der Gerade im Ursprünglichen Raum darstellen, müssen wir noch mit u multiplizieren:

$$x_{proj} = uu^T x$$

Die Projektionsmatrix ist hier also

$$P_1 = uu^T.$$

Möchten wir x stattdessen auf das orthogonale Komplement des von u aufgespannten Unterraums projizieren, müssen wir die Projektion auf u von x abziehen. Das bedeutet, in diesem Fall

$$x_{proj} = x - uu^T x = (I - uu^T)x.$$

Damit ist die Projektionsmatrix hier

$$P_2 = I - uu^T.$$

Möchten wir auf einen Unterraum projizieren, der von mehreren Vektoren $u_1, u_2, \dots, u_n$ aufgespannt wird, kann man diese Idee weniger einfach umsetzen. Wir verwenden dafür einen Least-Squares-Ansatz. Sei

$$U = \begin{bmatrix} | & | & \cdots & | \\ u_1 & u_2 & \cdots & u_n \\ | & | & \cdots & | \end{bmatrix}$$

Wir möchten Koordinaten y in der Basis von U derart finden, sodass

$$D(y) := \|x - Uy\|_2^2$$

minimal wird. Die Projektion x_{proj} wäre dann durch

$$x_{proj} = Uy$$

gegeben. Um das ideale y zu finden, setzen wir den Gradienten $\nabla_y D = 0$:

$$\nabla_y D = \nabla_y [(x - Uy)^T (x - Uy)] = 2U^T (Uy - x) = 0$$

$$U^T (Uy - x) = 0$$

$$U^T Uy = U^T x$$

$$y = (U^T U)^{-1} U^T x$$

Der Ausdruck $(U^T U)^{-1} U^T$ ist als Pseudo-Inverse bekannt. Für die Projektion erhalten wir schließlich

$$x_{proj} = Uy = U(U^T U)^{-1} U^T x$$

und die Projektionsmatrix ist

$$P_3 = U(U^T U)^{-1} U^T.$$

Wenn wir stattdessen auf das orthogonale Komplement von des von U aufgespannten Unterraums projizieren wollen, ist das ähnlich wie vorhin möglich:

$$P_4 = I - U(U^T U)^{-1} U^T$$

Übrigens zeigt sich für den Spezialfall $U = u$ (eindimensionaler Unterraum), dass $P_3 \rightarrow P_1$ und $P_4 \rightarrow P_2$.

6.3 Herleitung der PCA

Gegeben seien Datenpunkte $x_1, x_2, \dots, x_n \in \mathbb{R}^d$, deren Mittelwert $\mu = 0$ dem Nullvektor entspricht. Ist das nicht der Fall, 'normiert' man die Daten indem man einfach μ von den einzelnen Datenpunkten abzieht. Das Ziel ist es nun, die normierte Richtung v_1 ($\|v_1\|_2 = 1$) zu finden, sodass die empirische Varianz σ^2 der auf v_1 projizierten Daten maximal wird. Diese Varianz erhalten wir, weil v_1 normiert ist und $\mu = 0$, durch

$$\sigma^2(v_1) = \frac{1}{n} \sum_{k=1}^n (v_1^T x_k)^2.$$

Um die Summe in diesem Ausdruck loszuwerden, fassen wir die Daten in einer einzelnen Matrix

$$X = \begin{bmatrix} | & | & \cdots & | \\ x_1 & x_2 & \cdots & x_n \\ | & | & \cdots & | \end{bmatrix}$$

zusammen und schreiben stattdessen

$$\sigma^2(v_1) = \frac{1}{n} (v_1^T X) (v_1^T X)^T = \frac{1}{n} v_1^T X X^T v_1 = v_1^T C v_1.$$

Es stellt sich heraus, dass $C = \frac{1}{n} X X^T$ genau der Kovarianz-Matrix der Daten entspricht.

Die Richtung v_1 der maximalen Varianz ist dann jene, die die 'meiste Information' über die Daten enthält und ist die erste Hauptkomponente. Also optimieren wir nach v_1 unter der Nebenbedingung, dass $\|v_1\|_2 = 1$ bzw. $v_1^T v_1 = 1$, denn v_1 ist schließlich normiert. Nachdem das ein restringiertes Optimierungsproblem ist, nutzen wir die Lagrange-Multiplier aus dem vorherigen Abschnitt.

$$L(v_1, \lambda) = \sigma^2(v_1) + \lambda(v_1^T v_1 - 1)$$

Für die Ableitungen nach v_1 und λ erhalten wir mit

$$\frac{\partial L}{\partial \lambda} = v_1^T v_1 - 1 = 0$$

die Nebenbedingung und

$$\nabla_{v_1} L = (C + C^T)v_1 + 2\lambda v_1 = 0.$$

Weil C symmetrisch ist (Eigenschaft einer Kovarianzmatrix oder $C^T = \frac{1}{n} (X X^T)^T = \frac{1}{n} (X^T)^T X^T = \frac{1}{n} X X^T = C$) erhält man weiter die Gleichung

$$2Cv_1 = -2\lambda v_1$$

und mit $\lambda_1 := -\lambda$ die Eigenwertgleichung:

$$Cv_1 = \lambda_1 v_1.$$

Die optimale Richtung v_1 ist also ein Eigenvektor der Kovarianzmatrix (mit $\|v_1\|_2 = 1$). Nun stellt sich aber die Frage, welcher Eigenvektor das ist? Es gibt ja schließlich mehrere...

Antwort:

$$\begin{aligned} \sigma^2(v_1) &= v_1^T C v_1 \\ &= v_1^T \lambda_1 v_1 \\ &= \lambda_1 v_1^T v_1 \end{aligned}$$

und weil $v_1^T v_1 = 1$

$$= \lambda_1$$

Um die Varianz zu maximieren, müssen wir also den Eigenvektor mit dem größten Eigenwert der Kovarianzmatrix als Hauptkomponente wählen. Doch wie berechnet man die zweite, dritte usw. Hauptkomponente?

Es wäre durchaus plausibel, für die zweite Hauptkomponente alle Daten auf das orthogonale Komplement des Unterraums von v_1 zu projizieren (die Richtung v_1 damit quasi 'auszulöschen')

und dann die neue Richtung der maximalen Varianz (= die zweite Hauptkomponente v_2) durch das gleiche Optimierungsproblem wie vorhin zu bestimmen.

Zunächst die Projektion: Für die auf das orthogonale Komplement von v_1 projizierten Daten X_{proj} gilt nach Abschnitt 6.2:

$$X_{proj} = (I - v_1 v_1^T)X.$$

Für die neue Kovarianzmatrix C_{proj} erhalten wir

$$C_{proj} = X_{proj} X_{proj}^T = (I - v_1 v_1^T) X X^T (I - v_1 v_1^T)^T = (I - v_1 v_1^T) C (I - v_1 v_1^T),$$

denn die Projektionsmatrix $I - v_1 v_1^T$ ist symmetrisch.

Wenn wir jetzt dasselbe Optimierungsproblem wie vorhin (nur statt nach v_1 mit C nach v_2 mit C_{proj}) lösen, erhalten wir wieder die Nebenbedingung

$$v_2^T v_2 = 1$$

und die Eigenwertgleichung

$$C_{proj} v_2 = \lambda_2 v_2.$$

Weil $v_1 \perp v_2$ (durch die orthogonale Projektion) vereinfacht sich die Eigenwertgleichung folgendermaßen:

$$\begin{aligned} C_{proj} v_2 &= \lambda_2 v_2 \\ (I - v_1 v_1^T) C (I - v_1 v_1^T) v_2 &= \lambda_2 v_2 \\ I C I v_2 - v_1 v_1^T C I v_2 - I C v_1 v_1^T v_2 + v_1 v_1^T v_1 v_1^T v_2 &= \lambda_2 v_2 \\ C v_2 &= \lambda_2 v_2 \end{aligned}$$

Das bedeutet, dass v_2 auch wieder ein Eigenvektor von C sein muss - allerdings diesmal der mit dem zweitgrößten Eigenwert, denn sonst wäre $v_2 = v_1 \Rightarrow v_1 \not\perp v_2$.

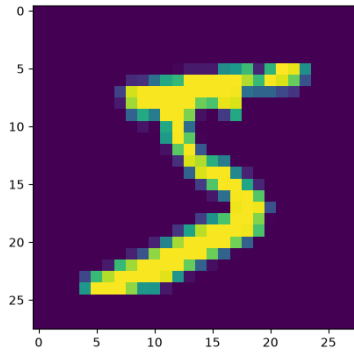
Allgemein kann man somit festhalten: Die k -te Hauptkomponente v_k ist der Eigenvektor von C mit dem k -t-größten Eigenwert. Es ist gar nicht notwendig, irgendeine Projektion durchzuführen. Alles was man tun muss, um die Hauptkomponenten der Daten zu erhalten, ist, die Eigenvektoren der Kovarianzmatrix zu bestimmen und diese absteigend nach zugehörigem Eigenwert zu ordnen.

Übrigens wissen wir aus Abschnitt 6.1, dass eine symmetrische Matrix immer zueinander orthogonale Eigenvektoren hat. Da C symmetrisch ist, sind auch alle Eigenvektoren von C orthogonal zueinander. Das passt genau damit zusammen, dass auch die Hauptkomponenten (die ja den Eigenvektoren entsprechen) alle orthogonal zueinander sind.

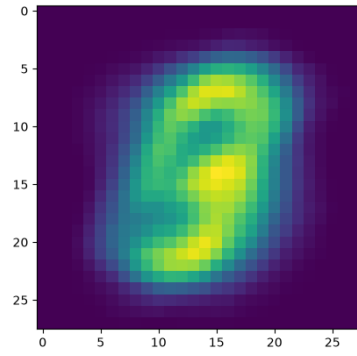
6.4 Anwendung der PCA auf den MNIST-Datensatz

Als praktisches Beispiel habe ich die PCA auf den MNIST-Datensatz (also auf 60000 mit Ziffern von 0 bis 9 galabelte 28×28 -Pixel-Bilder von handgeschriebenen Ziffern, Beispiel siehe Abbildung 5a) angewendet. Dabei repräsentiere ich zunächst jedes Bild als 784-dimensionalen Vektor. Als erstes habe ich dann die Daten normiert, indem ich den Mittelwert (also das 'durchschnittliche Bild', siehe Abbildung 5b) von jedem Bild in den Daten subtrahiert habe. Dann habe ich die Kovarianzmatrix C bestimmt. Die Eigenvektoren erhalte ich einfach durch den entsprechenden NumPy-Befehl und kann sie entsprechend ihrer Eigenwerte sortieren. Anschließend wähle ich die zwei wichtigsten Eigenvektoren aus und nutze sie als Hauptkomponenten der Daten. Diese 'Eigen-Bilder' sind in Abbildung 6 zu sehen.

Dann habe ich noch einen Teil der Daten (10000 Bilder) auf diese beiden Hauptkomponenten projiziert und die projizierten Punkte als Punktwolke dargestellt (Abbildung 7). Dabei habe ich die Punkte mit dem gleichen Label (also die zu Bildern gehören, die dieselbe Ziffer anzeigen) mit jeweils derselben Farbe eingefärbt. Interessanterweise sieht man, dass sich die Projektionen der Bilder auf die Hauptkomponenten je nach Ziffer unterscheiden und dadurch sichtbare Cluster entstehen. Dasselbe habe ich auch noch für drei Dimensionen gemacht und eine Animation erstellt (allerdings mit weniger Datenpunkten damit es sich nicht zu sehr aufhängt).

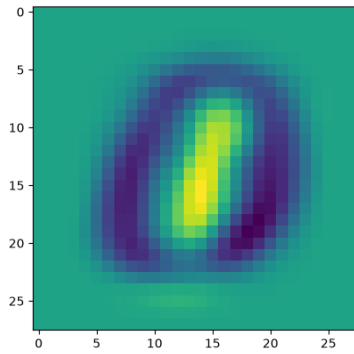


(a) Beispiel für ein Bild aus dem MNIST-Datensatz

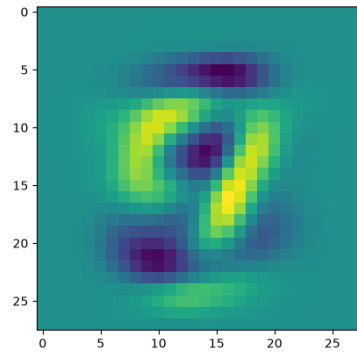


(b) Durchschnittliches MNIST-Bild

Abbildung 5: Exemplarisches und durchschnittliches MNIST-Bild



(a) Erste Hauptkomponente



(b) Zweite Hauptkomponente

Abbildung 6: 'Eigen-Bilder' bzw. die ersten beiden Hauptkomponenten

7 t-Stochastic Neighbor Embedding (t-SNE)

Ebenso wie die PCA ist auch das t-SNE eine Methode der Dimensionenreduktion von Daten. Allerdings ist das t-SNE eher nur zur Veranschaulichung gedacht, da es zwar bessere Visualisierungen liefert als andere Methoden wie z.B. die PCA (wie in Abb. 7), jedoch keine gute Interpretierbarkeit (wie z.B. die Sache mit den Eigenwerten) besitzt.

Beim t-SNE erhalten hochdimensionale Daten $\mathcal{X} = \{x_1, x_2, \dots, x_n\}$ je eine niedrigerdimensionale (d.h. 2D oder 3D) Repräsentation $\mathcal{Y} = \{y_1, y_2, \dots, y_n\}$. Diese wird zunächst zufällig festgelegt und anschließend optimiert, sodass die paarweisen Abstände der einzelnen Punkte zueinander (gemessen als Wahrscheinlichkeiten) bei der Zuordnung von $\mathcal{X}$ zu $\mathcal{Y}$ möglichst unverändert bleiben. So werden diese Abstandsmaße eingeführt:

Gegeben seien zwei Datenpunkte x_i und x_j . Die Wahrscheinlichkeit $p_{i|j}$, dass x_i den Punkt x_j als Nachbarn 'wählt' wird modelliert durch

$$p_{i|j} = \begin{cases} \frac{\exp(-\|x_i - x_j\|_2^2 / (2\sigma^2))}{\sum_{k=1, k \neq i}^n \exp(-\|x_i - x_k\|_2^2 / (2\sigma^2))}, & \text{falls } i \neq j \\ 0 & \text{falls } i = j \end{cases}.$$

Um die Ähnlichkeit (bzw. den Abstand) $p_{i,j}$ zwischen den beiden Datenpunkten zu messen, nutzt man die Formel

$$p_{i,j} = \frac{p_{i|j} + p_{j|i}}{2n}$$

damit das Ähnlichkeitsmaß symmetrisch wird (also $p_{i,j} = p_{j,i}$).

Ein weiteres Ähnlichkeitsmaß $q_{i,j}$ definieren wir nun für die niedrigdimensionalen Bilder y_i, y_j

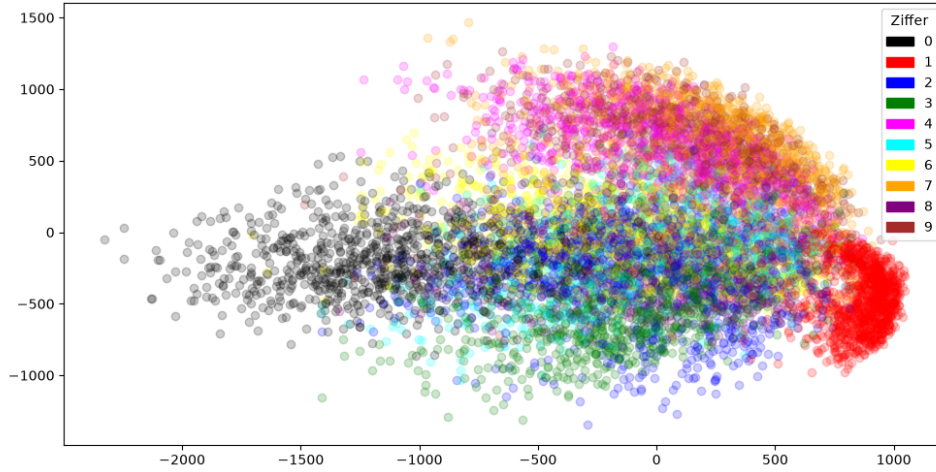


Abbildung 7: 10000 Bilder des MNIST-Datensatzes auf die ersten beiden Hauptkomponenten projiziert, Darstellung einzelner Ziffern durch unterschiedliche Farben

von x_i und x_j :

$$q_{i,j} = \begin{cases} \frac{(1 + \|y_i - y_j\|_2^2)^{-1}}{\sum_{k,l=1, k \neq l}^n (1 + \|y_l - y_k\|_2^2)^{-1}}, & \text{falls } i \neq j \\ 0 & \text{falls } i = j \end{cases}$$

Während das Ähnlichkeitsmaß im höherdimensionalen Raum an eine Normalverteilung erinnert, ist die Ähnlichkeit im Niedrigdimensionalen von der Student-t-Verteilung inspiriert. Daher kommt auch das 't' in 't-SNE'.

Ziel ist es nun, Repräsentationen y so zu finden, sodass die beiden Ähnlichkeitsmaße einander möglichst genau entsprechen. Die Abweichung zwischen P und Q ist deren KL-Divergenz:

$$D_{KL}(P||Q) = \sum_{i,j=1, i \neq j}^n p_{i,j} \log \frac{p_{i,j}}{q_{i,j}}$$

Wir möchten diese also minimieren - und das geschieht mittels Gradient Descent. Im speziellen nutzen wir die Polyak's Heavy Ball Methode, also Gradient Descent mit einem Momentum-Term. Es stellt sich zudem heraus, dass man den Gradienten der Kullback-Leibler Divergenz symbolisch bestimmen kann und er ist durch

$$\nabla_{y_i} D_{KL} = 4 \sum_{j=1}^n \frac{p_{i,j} - q_{i,j}}{1 + \|y_i - y_j\|_2^2} (y_i - y_j)$$

gegeben. Die optimalen Werte für die Darstellungen der Daten in $\mathcal{Y}$ werden schließlich genutzt, um die Daten im niedrigerdimensionalen Raum darzustellen.

Ich habe diesen Vorgang für einfache bzw. kleine Datensätze in Python implementiert. Dabei ist für ein Beispiel mit 20 6-dimensionalen Datenpunkten, die von zwei voneinander verschiedenen Normalverteilungen gesampelt wurden, die in Abbildung 8a gezeigte 2D-Darstellung herausgekommen. Leider ist meine Implementierung sehr ineffizient, da sie auf vielen (in Python langsamen) Schleifen beruht. Dadurch wird es unmöglich mit größeren Datensätzen zu arbeiten. Um dennoch den MNIST-Datensatz von vorhin zum Vergleich mit t-SNE darstellen zu können, habe ich statt meiner Lösung Florians Musterlösung genutzt, die auf wesentlich schnellere (aber auch kompliziertere) Operationen mit NumPy-Arrays zurückgreift. Das Ergebnis für die Ziffern 0 bis 4 ist in Abbildung 8b zu sehen.

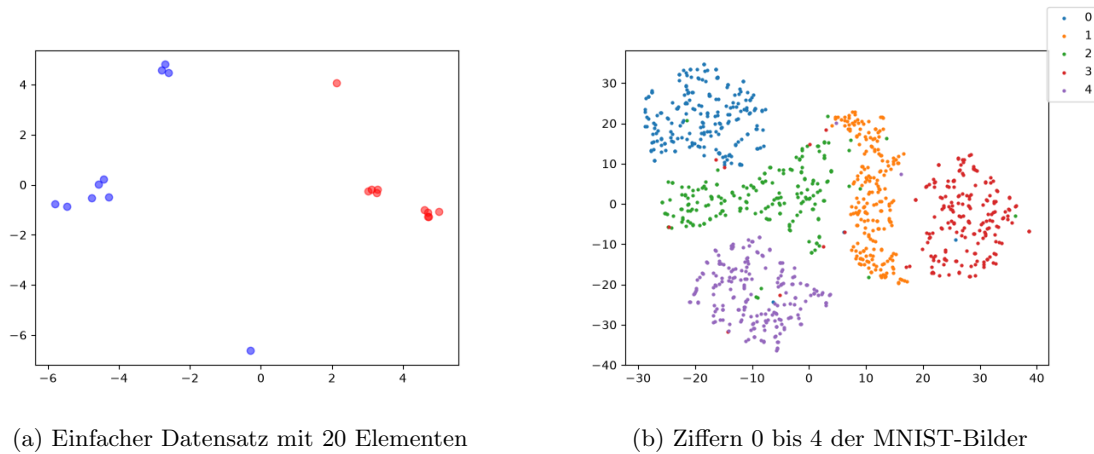


Abbildung 8: Veranschaulichung von Daten mittels t-SNE anhand von zwei Beispielen

8 Lineare Programme

Lineare Programme sind Optimierungsprobleme, bei denen eine lineare Zielfunktion $z = c^T x$ unter linearen Gleichungs- und Ungleichungseinschränkungen (nur mit $\geq$ und $\leq$) maximiert oder minimiert werden soll.

8.1 Allgemeine Formulierung

Damit lineare Programme besser vereinheitlicht werden können, schreibt man sie in der Form

$$\max c^T x \text{ unter der Bedingung, dass } Ax \leq b$$

an, wobei das $\leq$ komponentenweise definiert ist. Jedes lineare Programm kann unter der Anwendung folgender Tricks in diese Form umgewandelt werden:

1. Minimierungsprobleme können einfach in ein Maximierungsproblem umgewandelt werden, indem man statt $c^T x$ zu minimieren, $-c^T x$ maximiert.
2. Gleichungen können als zwei Ungleichungen umformuliert werden. Zum Beispiel ist die Gleichung $x_1 + x_2 = 2$ gleichwertig mit dem Ungleichungspaar aus $x_1 + x_2 \geq 2$ und $x_1 + x_2 \leq 2$.
3. Ungleichungen mit einem ' $\geq$ ' können in eine Ungleichung mit einem ' $\leq$ ' umgeformt werden, indem man mit -1 multipliziert. Beispiel: $x_1 + x_2 \geq 2 \implies -x_1 - x_2 \leq -2$
4. Selbst Bedingungen der Form $|a^T x| \leq b$ (also mit einem Betrag) können in zwei Ungleichungen zerlegt werden, denn $|a^T x| \leq b \iff a^T x \leq b \text{ und } -a^T x \leq b$.

8.2 Lösungsfälle von Linearen Programmen

Lineare Programme können entweder eine Lösung, unendlich viele Lösungen oder keine Lösung haben. Dafür, dass ein lineares Programm keine Lösung besitzt, kann es mehrere Gründe geben. Entweder gibt es kein x , das widersprüchliche Bedingungen $Ax \leq b$ erfüllen könnte, oder das Programm ist unbeschränkt; das heißt dass keine Lösung x gefunden werden kann, da es immer ein weiteres x gibt für das die Zielfunktion einen noch größeren Wert annimmt. Das bedeutet, dass die Lösung erst im Unendlichen zu finden wäre, man praktisch also keine Lösung hat.

8.3 Standard-Form eines LP

Die Standard-Form eines linearen Programms ist

$$\max c^T x \text{ mit } Ax = b, x \geq 0,$$

wobei das ' $\geq$ ' wieder komponentenweise definiert ist und die einzelnen Gleichungen (also die Zeilen von A linear unabhängig sind). Diese Formulierung wird benötigt, um den Simplex-Algorithmus,

der ein Lösungsverfahren für lineare Programme ist, darauf anwenden zu können. Man kann jedes LP in die Standardform übersetzen, muss dabei allerdings kreativ sein. Dafür gibt es wieder ein paar Tricks:

1. Eine Ungleichung wird zu einer Gleichung, indem man eine zusätzliche nicht-negative 'Slack'-Variable einführt. Beispielsweise kann man die Ungleichung $x_1 + x_2 \leq 2$ als $x_1 + x_2 + x_3 = 2$ schreiben mit $x_3 \geq 0$. Somit wird das gesamte Gleichungssystem $Ax + b$ um eine Spalte von A größer und x wächst um eine Dimension, aber das ist notwendig um mit dem Simplex-Algorithmus fortfahren zu können.
2. Es kann sein, dass die ursprünglichen Variablen des linearen Programms nicht zwingend als nicht-negative Zahlen definiert sind. Allerdings muss die Bedingung $x \geq 0$ für alle x_i in der Standardform erfüllt sein. Angenommen, die Variable x_1 kommt in zwar in dem linearen Programm vor, ist aber nicht durch $x_1 \geq 0$ eingeschränkt, dann ersetzt man x_1 in allen Gleichungen durch zwei neue Variablen $x^+ \geq 0$ und $x^- \geq 0$ mit $x_1 = x^+ - x^-$, denn jede reelle Zahl kann als Differenz zweier nicht-negativer reeller Zahlen dargestellt werden.

8.4 Zulässige Lösungen und Basislösungen

Sei $\max c^T x$ mit $Ax = b, x \geq 0$ ein lineares Programm in Standard-Form mit k Gleichungen und n Variablen, $k \leq n$ (d.h. $A \in \mathbb{R}^{k \times n}$). Dann heißt jedes x , für das $Ax = b$ und $x \geq 0$ erfüllt sind, 'zulässige Lösung'.

Definieren wir zudem für eine Menge $B \subseteq \{1, 2, \dots, n\}$ mit $|B| = k$ die Matrix A_B als jene Matrix, deren Spalten den Spalten von A mit einem Spaltenindex in B entspricht. Dann heißt jedes x , für das

1. $Ax = b$,
2. $\text{rk}(A_B) = k$ (d.h. $\det A_B \neq 0$) und
3. $\forall i \notin B : x_i = 0$

Basislösung des linearen Programms. Eine Basislösung, bei der mindestens ein Eintrag $x_i, i \in B$ gleich Null ist, heißt *entartete Basislösung*. Eine Lösung, die sowohl zulässig, als auch eine Basislösung ist, heißt *zulässige Basislösung*.

8.5 Fundamentalsatz der Linearen Programmierung

Der Fundamentalsatz der linearen Programmierung besagt, dass für ein lineares Programm in der Standard-Form, sofern es eine zulässige Lösung gibt, immer eine zulässige Basislösung existiert. Außerdem, wenn es eine optimale zulässige Lösung gibt, dann gibt es eine optimale zulässige Basislösung.

Beweis. Beseisen wir zunächst den ersten Teil.

Nehmen wir an, dass eine zulässige Lösung $x = (x_1, x_2, \dots, x_n)$ existiert, von der genau $0 \leq p \leq n$ Komponenten größer als Null sind, der Rest ist gleich Null. Nehmen wir zudem ohne Verlust der Allgemeinheit an, dass diese p positiven Einträge genau x_1, x_2 , usw. bis x_p sind. Seien die Spalten von A die Vektoren $a_1, a_2, \dots, a_n$. Dann gilt weil x zulässig ist

$$x_1 a_1 + x_2 a_2 + \dots + x_p a_p = b.$$

Es gibt nun zwei mögliche Fälle, die eintreten können: Entweder $a_1, a_2, \dots, a_p$ sind linear unabhängig (1) oder sie sind linear abhängig (2).

1. Wenn $a_1, a_2, \dots, a_p$ linear unabhängig sind, dann ist $p \leq k$. Falls $p = k$, dann ist $\text{rk}(A_{\{1, 2, \dots, p\}}) = k$, wodurch x eine Basislösung ist und der Beweis fertig ist. Andernfalls, wenn $p < k$, dann kann man aus den verbleibenden $n - p$ Spalten weitere $k - p$ Spalten auswählen, sodass diese Spalten linear unabhängig sind. Auch in diesem Fall ist x eine (entartete) Basislösung.
2. Wenn $a_1, a_2, \dots, a_p$ linear abhängig sind, dann existiert eine nichttriviale (d.h. nicht alle sind Null) Kombination von $y_1, y_2, \dots, y_p$, wobei mindestens eines dieser y größer als Null ist, sodass

$$y_1 a_1 + y_2 a_2 + \dots + y_p a_p = 0.$$

Somit gilt automatisch

$$(x_1 - \varepsilon y_1)a_1 + (x_2 - \varepsilon y_2)a_2 + \cdots + (x_p - \varepsilon y_p)a_p = b$$

für alle ε . Auch ist für alle ε

$$x - \varepsilon y$$

mit $y = (y_1, \dots, y_p, 0, \dots, 0)$ eine Lösung für das Gleichungssystem. Wählt man nun $\varepsilon = \min\{\frac{x_i}{y_i} : y_i > 0\}$ (es existiert mindestens ein solches $y_i > 0$), dann ist $x - \varepsilon y$ ebenfalls eine zulässige Lösung, die allerdings nicht mehr p sondern nur noch $p - 1$ von Null verschiedene Komponenten hat. Diesen Prozess kann man nun iterativ fortführen (d.h. mit $x \leftarrow x - \varepsilon y$ und neuem y), bis die entsprechenden Spalten von A linear unabhängig sind. Dann kann man Fall (1) anwenden.

Das bedeutet also, dass in jedem Fall aus einer zulässigen Lösung eine zulässige Basislösung konstruiert werden kann. Der erste Teil des Satzes ist also bewiesen.

Beweisen wir nun den zweiten Teil. Hier nehmen wir an, dass $x = (x_1, x_2, \dots, x_n)$ eine optimale zulässige Lösung ist und dass die ersten p Einträge in diesem Vektor wieder positiv sind, die darauffolgenden alle Null (wie vorhin). Dann gibt es wieder zwei Fälle. Die Spalten $a_1, a_2, \dots, a_p$ können entweder linear unabhängig (1) oder linear abhängig (2) sein.

1. Falls die Spalten linear unabhängig sind, kann man wie zuvor im ersten Teil des Beweises zeigen, dass x eine Basislösung ist.
2. Sind die Spalten linear abhängig, so kann man auch wie im ersten Teil eine Basislösung daraus konstruieren. Dabei sei angemerkt, dass der Optimalwert $c^T(x - \varepsilon y) = c^T x - \varepsilon c^T y = c^T x$ sein muss (denn sonst wäre x ja nicht optimal) und da ε sowohl positiv als auch negativ gewählt sein kann, muss zwangsläufig $c^T y = 0$. Das bedeutet, dass man aus x ebenso wie vorhin eine optimale zulässige Basislösung konstruieren kann.

□

Dieser Satz zeigt, dass man, um ein lineares Programm zu lösen, lediglich eine optimale zulässige Basislösung zu finden braucht. Da es nur maximal $\binom{n}{k}$ zulässige Basislösungen geben kann, reduziert sich das Optimierungsproblem von unendlich vielen zulässigen Lösungen auf eben diese endliche Anzahl potentieller Lösungen. Ein möglicher (aber sehr ineffizienter) Weg, ein lineares Programm zu lösen, wäre alle zulässigen Basislösungen durchzuprobieren. Der Simplex-Algorithmus hingegen bietet eine wesentlich strukturiertere und schnellere Methode, um die zulässigen Basislösungen abzusuchen und zu einer optimalen Lösung zu gelangen.

8.6 Simplex-Algorithmus

Den Simplex-Algorithmus möchte ich an einem Beispiel demonstrieren. Dieses lautet:

$$\max 2x_1 + x_2$$

unter den Bedingungen, dass

$$\begin{aligned} x_1 + x_2 &\leq 5, \\ 2x_1 + x_2 &\leq 8, \\ x_2 &\geq 0. \end{aligned}$$

Damit ich den Simplex-Algorithmus anwenden kann, muss ich die Nebenbedingungen zuerst als Gleichungen formulieren. Für die ersten beiden Bedingungen führe ich Slack-Variablen ein. Die neuen Nebenbedingungen lauten also:

$$\begin{aligned} x_1 + x_2 + x_3 &= 5, \\ 2x_1 + x_2 + x_4 &= 8, \\ x_2, x_3, x_4 &\geq 0. \end{aligned}$$

Nachdem x_1 eine freie Variable ist (also auch negative Werte annehmen darf), muss ich diese durch weitere zwei Variablen ersetzen. Die Substitution lautet $x_1 = x_5 - x_6, x_{5,6} \geq 0$ und für die Nebenbedingungen ergibt sich

$$\begin{aligned} x_2 + x_3 + x_5 - x_6 &= 5, \\ x_2 + x_4 + 2x_5 - 2x_6 &= 8, \\ x_2, x_3, x_4, x_5, x_6 &\geq 0. \end{aligned}$$

Das ist ein LP in Standardform, denn man kann es als

$$\max c^T x \text{ unter der Bedingung, dass } Ax = b, x \geq 0$$

anschreiben, wobei

$$A = \begin{bmatrix} 1 & 1 & 0 & 1 & -1 \\ 1 & 0 & 1 & 2 & -2 \end{bmatrix}, b = \begin{bmatrix} 5 \\ 8 \end{bmatrix}, c = [1 \quad 0 \quad 0 \quad 2 \quad -2]^T$$

und $x = (x_2, x_3, x_4, x_5, x_6)^T$. Nun kann man den Simplex-Algorithmus auf das Optimierungsproblem anwenden. Dazu benötigt man zuallererst eine zulässige Basislösung, von der ausgehend man starten (d.h. bessere Lösungen finden) kann. Eine offensichtliche zulässige Basislösung ist diesem

Fall $x_0 = (0, 5, 8, 0, 0)^T$, da $Ax_0 = b$, $\det(A_{\{2,3\}}) = \det\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = 1 \neq 0$ und $x_0^{(i)} = 0 \quad \forall i \notin \{2, 3\}$.

Das LP kann man jetzt in einem sogenannten Simplex-Tableau festhalten:

$$\begin{array}{rcll} x_3 & = & 5 & -x_2 & -x_5 & +x_6 \\ x_4 & = & 8 & -x_2 & -2x_5 & +2x_6 \\ \hline z & = & & x_2 & +2x_5 & -2x_6 \end{array}$$

Die 'aktuelle' Lösung für die Zielvariable ist 0, weil $x_2 = x_5 = x_6 = 0$. Um z zu vergrößern (wir wollen es ja maximieren), ist es notwendig, sich anzuschauen, welche x_i man verändern kann, damit z größer wird. Nachdem alle Variablen positiv definiert sind, macht es nur Sinn x_2 oder x_5 zu erhöhen; eine Erhöhung von x_6 würde zu einer nicht wünschenswerten Senkung von z führen. Ich wähle jetzt einfach x_2 , wobei es aber genauso möglich wäre, x_5 zu wählen. Jetzt stellt sich nur die Frage, um wie viel man x_2 erhöhen darf. Es stellt sich wegen der ersten Zeile heraus, dass man x_2 um maximal 5 erhöhen darf, weil ansonsten $x_3 < 0$ wäre, was nicht zulässig ist. Tauschen wir also x_2 und x_3 aus. Es gilt:

$$x_2 = 5 - x_3 - x_5 + x_6$$

Setzt man das in die Tabelle von oben ein, erhält man folgendes neues Tableau:

$$\begin{array}{rcll} x_2 & = & 5 & -x_3 & -x_5 & +x_6 \\ x_4 & = & 3 & +x_3 & -x_5 & +x_6 \\ \hline z & = & 5 & -x_3 & +x_5 & -x_6 \end{array}$$

Der aktuelle Wert der Zielfunktion ist 5. Um diesen weiter zu erhöhen, kann man jetzt nur noch x_5 (um 3 wegen Gl. 2) steigern.

$$x_5 = 3 + x_3 - x_4 + x_6$$

Das neue und gleichzeitig letzte Tableau ist

$$\begin{array}{rcll} x_2 & = & 2 & -2x_3 & +x_4 \\ x_5 & = & 3 & +x_3 & -x_4 & +x_6 \\ \hline z & = & 8 & & & -x_4 \end{array}$$

da man z nicht weiter erhöhen kann. Die Optimale Lösung ist damit $x = (2, 0, 0, 5, 0)^T$ und der Maximalwert der Zielfunktion ist 8.

Diese Lösung kann man jetzt für das originale LP nutzen. Die Lösung des ursprünglichen LP ist

$$x_1 = x_5 - x_6 = 3, x_2 = 2 \text{ und } 2x_1 + x_2 = 8.$$

8.7 Probleme bzw. Sonderfälle beim Simplex-Algorithmus

Das erste Problem, das beim Lösen eines linearen Programms auftreten kann ist, dass es zu Beginn keine offensichtliche Basislösung gibt, die als Ausgangspunkt für den Simplex-Algorithmus dient. Um dennoch eine solche zulässige Ausgangs-Basislösung finden zu können, müssen wir zunächst ein weiteres LP lösen. Ich demonstriere das wieder an einem Beispiel:

Gegeben sei das LP

$$\max x_1 + x_2$$

mit

$$\begin{aligned}x_1 + 2x_2 &\geq 4 \\2x_1 - x_2 &\leq 5 \\x_1, x_2 &\geq 0\end{aligned}$$

Durch Slack-Variablen formen wir das LP in die Standardform um:

$$\max x_1 + x_2$$

mit

$$\begin{aligned}x_1 + 2x_2 - x_3 &= 4 \\2x_1 - x_2 + x_4 &= 5 \\x_1, x_2, x_3, x_4 &\geq 0\end{aligned}$$

Da man keine offensichtliche Basislösung ablesen kann, benötigen wir stattdessen eine Methode, eine solche Basislösung systematisch zu finden. Dann fügen wir in jede Gleichung eine weitere künstliche Variable (x_5 und x_6) ein, sodass die Nebenbedingungen

$$\begin{aligned}x_1 + 2x_2 - x_3 + x_5 &= 4 \\2x_1 - x_2 + x_4 + x_6 &= 5 \\x_1, x_2, x_3, x_4, x_5, x_6 &\geq 0\end{aligned}$$

lauten. Minimieren wir nun $x_5 + x_6$ (bzw. wir maximieren $-x_5 - x_6$) mithilfe des Simplex-Algorithmus. Als Start-Basis wählen wir dazu $x_5 = 4, x_6 = 5$. Immer, wenn man den eben beschriebenen 'Trick' anwendet, ist eine solche Wahl möglich. Wenn eine zulässige Basislösung für das ursprüngliche Optimierungsproblem existiert, hat die Zielfunktion dieses erweiterten Problems den Wert Null. Ist der Optimalwert ungleich Null, dann existiert keine einzige zulässige Basislösung für das ursprüngliche LP und es ist nicht lösbar. Übrigens existiert für dieses erweiterte Optimierungsproblem immer eine Lösung, weil die Zielfunktion ($-x_5 - x_6$) niemals größere Werte als Null annehmen kann und somit beschränkt ist. Hier ist das erweiterte Programm mit Simplex-Tableaus einmal durchgerechnet:

$$\max -x_5 - x_6$$

unter den Bedingungen, dass

$$\begin{aligned}x_1 + 2x_2 - x_3 + x_5 &= 4 \\2x_1 - x_2 + x_4 + x_6 &= 5 \\x_1, x_2, x_3, x_4, x_5, x_6 &\geq 0\end{aligned}$$

ergibt als Simplex-Tableau (mit $z = -x_5 - x_6 = -9 + 3x_1 + x_2 - x_3 + x_4$)

$$\begin{array}{rcllcl}x_5 & = & 4 & -x_1 & -2x_2 & +x_3 \\x_6 & = & 5 & -2x_1 & +x_2 & -x_4. \\ \hline z & = & -9 & +3x_1 & +x_2 & -x_3 +x_4\end{array}$$

Nutzen wir nun x_2 als eintretende Basisvariable, dann muss x_5 die Basis verlassen und das nächste Tableau ist:

$$\begin{array}{rcllcl}x_2 & = & 2 & -0.5x_1 & +0.5x_3 & -0.5x_5 \\x_6 & = & 7 & -2.5x_1 & +0.5x_3 & -x_4 -0.5x_5 \\ \hline z & = & -7 & +2.5x_1 & -0.5x_3 & +x_4 -0.5x_5\end{array}$$

Nächster Schritt, x_4 wird zur Basisvariable, x_6 verlässt die Basis:

$$\begin{array}{rcllcl}x_2 & = & 2 & -0.5x_1 & +0.5x_3 & -0.5x_5 \\x_4 & = & 7 & -2.5x_1 & +0.5x_3 & -0.5x_5 -x_6 \\ \hline z & = & 0 & & & -x_5 -x_6\end{array}$$

Das Lösen des erweiterten LP ergibt in diesem Fall $x_2 = 2, x_4 = 7$ und $z = 0$, was tatsächlich eine zulässige Basislösung ist für das ursprüngliche LP ist. Nun kann dieses LP einfach mit dem Simplex-Algorithmus gelöst werden. Diese soeben beschriebene Vorgehensweise wird auch Phase I des Simplex-Algorithmus genannt, weil sie in den meisten Fällen notwendig ist. Das eigentliche Lösen des LP ausgehend von der in Phase I gewonnenen zulässigen Start-Basislösung wird dann auch als Phase II bezeichnet.

Allerdings gibt es noch ein Problem, das beim Lösen eines linearen Programms auftreten kann. Nämlich - wie zuvor schon angedeutet - kann es vorkommen, dass das LP unbeschränkt ist, die

optimale Lösung also unendlich groß wäre. Als Beispiel dafür dient das vorherige zusammen mit der eben ausgerechneten Start-Basislösung:

Das ursprüngliche LP (mit den Slack-Variablen) lautet

$$\max c^T x \text{ unter den Bedingungen, dass } Ax = b, x \geq 0$$

mit

$$A = \begin{bmatrix} 1 & 2 & -1 & 0 \\ 2 & -1 & 0 & 1 \end{bmatrix}, b = \begin{bmatrix} 4 \\ 5 \end{bmatrix}, c = [1 \quad 1 \quad 0 \quad 0]^T$$

Zudem haben wir jetzt schon eine zulässige Basislösung gefunden, nämlich

$$x_0 = (0, 2, 0, 7)^T, B = \{2, 4\}$$

Damit das LP in ein Simplex-Tableau eingetragen werden kann, formen wir die Nebenbedingung $Ax = b$ um:

$$Ax = b \rightarrow A_B^{-1} Ax = A_B^{-1} b$$

Es gilt

$$A_B^{-1} A = \begin{bmatrix} 0.5 & 1 & -0.5 & 0 \\ 2.5 & 0 & -0.5 & 1 \end{bmatrix},$$

wobei $(A_B^{-1} A)_B$ der Identitätsmatrix entspricht und

$$A_B^{-1} b = \begin{bmatrix} 2 \\ 7 \end{bmatrix} = x_0$$

Somit kann man das LP offensichtlich folgendermaßen in ein Simplex-Tableau bringen:

$$\begin{array}{rcll} x_2 & = & 2 & -0.5x_1 & +0.5x_3 \\ x_4 & = & 7 & -2.5x_1 & +0.5x_3 \\ \hline z & = & & 0.5x_1 & +0.5x_3 \end{array}$$

Man könnte nun sowohl x_1 als auch x_3 erhöhen, um zum Maximum von z zu gelangen - allerdings gibt es dabei ein Problem. Denn wenn wir x_3 erhöhen wollen, gibt es keine Einschränkung dafür, um wie viel man x_3 erhöhen darf, weil ein beliebig großes x_3 gegen keine Nebenbedingung verstößt. Es ist somit nicht beschränkt und das LP hat keine optimale Lösung (oder eben $z = \infty$).

8.8 Implementation des Simplex-Algorithmus in Python

Ich habe den Simplex-Algorithmus (mit Phase I und II) in Python implementiert. Dabei ist bei meiner Implementation wichtig anzumerken, dass das Programm Lineare Programme nur in Standardform mit $b \geq 0$ lösen kann und es theoretisch möglich wäre, dass es sich aufhängt, denn es verwendet keine Entscheidungshilfen für die einzelnen Pivot-Schritte. Abgesehen davon funktioniert die Implementation aber einwandfrei.

Die Implementation beinhaltet grundlegende Funktionen wie etwa die Feststellung, ob eine Lösung zulässig oder eine Basislösung ist, einen Solver, der Basislösungen und auch die optimale Lösung durch Probieren ermitteln kann (sehr langsam) und einen Solver, der den Simplex-Algorithmus anwendet. Der Simplex-Algorithmus ist als Aneinanderreihung von Pivot-Steps umgesetzt, als Methode eines `SimplexTable`-Objekts definiert ist. Das `SimplexTable`-Objekt kann aus A , b , c und einer zulässigen Basislösung mit deren Basis (Menge von Indizes) erstellt werden und speichert die einzelnen Teile des Simplex-Tableaus ab. Die Methode `pivot_step()` von `SimplexTable` kann nicht nur (falls möglich) einen Pivot-Schritt ausführen, sondern auch erkennen, wenn das nicht möglich ist (weil der Algorithmus bereits am Optimum angelangt ist oder weil das Problem unbeschränkt ist).

Schließlich gibt es noch die Funktion `solve_LP()`, die bei gegebenen A , $b \geq 0$ und c eine Nachricht über den Lösungsfall (d.h. Existenz einer optimalen Lösung, Unbeschränktheit, Unzulässigkeit), den (oder im Falle mehrerer optimaler Lösungen einen) optimalen Wert für x sowie das Maximum von z zurückgibt.

8.9 Anwendungsbeispiel: Task Scheduling mit beliebig teilbaren Tasks

Gegeben seien m Maschinen (oder Arbeiter) und t verschiedene Tasks bzw. Aufgaben. Zudem gibt es eine Matrix $T \in \mathbb{R}^{t \times m}$, deren Einträge T_{ij} beschreiben, wie lange Maschine j für eine Ausführung von Task i benötigt. Außerdem gibt es noch einen Vektor $w \in \mathbb{R}^t$, der beschreibt, wie oft die einzelnen Tasks ausgeführt werden müssen. Schließlich nehmen wir der Einfachheit wegen an, dass Tasks beliebig unterteilt werden können, also dass z.B. Maschine 1 die Hälfte eines Tasks und Maschine 2 die andere Hälfte übernimmt. Dadurch wird verhindert, dass man ein LP mit nur ganzzahligen zulässigen Lösungen lösen muss, was deutlich komplizierter ist. Das Ziel ist es jetzt, die gesamte Laufzeit zu minimieren, wobei die Maschinen parallel arbeiten. Dabei wollen wir die Matrix $W \in \mathbb{R}^{t \times m}$ ermitteln, die in der i -ten Zeile und der j -ten Spalte beinhaltet, wie viel Maschine j von Task i übernimmt.

Dafür nutzen wir folgende Überlegungen. Zunächst muss für alle $i = 1, 2, \dots, t$ gelten, dass

$$\sum_{j=1}^m W_{ij} = w_i,$$

also, dass die Maschinen insgesamt die richtigen Anzahlen der einzelnen Tasks erledigen. Außerdem muss

$$W_{ij} \geq 0$$

weil keine Maschine eine negative Anzahl an Tasks erledigen kann. Schließlich nehmen wir noch an, dass alle Maschinen gleich lange arbeiten, weil Maschinen sonst nichts tun könnten, während andere noch Arbeit erledigen, was sehr ineffizient wäre. Also muss für alle $j = 1, \dots, m$

$$\sum_{i=1}^t T_{i1} W_{i1} - \sum_{i=1}^t T_{ij} W_{ij} = 0.$$

Das sind die Nebenbedingungen.

Das Ziel des Optimierungsproblems ist es, die Laufzeit, also die Zeit, die die Maschine mit der längsten Arbeitszeit benötigt, zu minimieren, d.h. das Optimierungsziel würde

$$\min_W \max_j \sum_{i=1}^t T_{ij} W_{ij}$$

lauten. Da allerdings alle Maschinen dieselbe Zeit lange arbeiten, entspricht die maximale Arbeitszeit der durchschnittlichen Arbeitszeit. Das Optimierungsziel ist also

$$\min_W \frac{1}{m} \sum_{j=1}^m \sum_{i=1}^t T_{ij} W_{ij}.$$

Zur Verdeutlichung werde ich diese allgemeine Formulierung auf ein konkretes Beispiel anwenden. Es gibt zwei Maschinen A und B sowie zwei Tasks $T1$ und $T2$. Maschine A kann $T1$ in einer Stunde erledigen und braucht für $T2$ zwei Stunden. B hingegen benötigt für $T1$ zwei Stunden, ist dafür aber in einer Stunde mit $T2$ fertig. Insgesamt muss $T1$ genau fünf Mal erledigt werden und $T2$ genau zwei Mal. Wir suchen die optimale Arbeitsaufteilung für die Maschinen A und B .

Für die Matrix T und den Vektor w erhält man

$$T = \begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix}, w = \begin{bmatrix} 5 \\ 2 \end{bmatrix}.$$

Die unbekannten Variablen $W = \begin{bmatrix} W_{00} & W_{01} \\ W_{10} & W_{11} \end{bmatrix}$ fasse ich als nächstes in einen Vektor $x := (W_{00}, W_{01}, W_{10}, W_{11})^T$ zusammen. Das Ziel ist es nun,

$$\frac{1}{2} c^T x$$

mit $c = (1, 2, 2, 1)^T$ zu minimieren, unter den Bedingungen, dass

$$Ax = b$$

mit $A = \begin{bmatrix} 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 1 & -2 & 2 & -1 \end{bmatrix}$ und $b = (5, 2, 0)^T$ und, dass

$$x \geq 0$$

Das angegebene Gleichungssystem folgt aus den Nebenbedingungen. Es zeigt sich, dass das genau die Formulierung eines linearen Programms in Standard-Form ist. Lösen des LP gibt in diesem Fall die optimale Arbeitsaufteilungsmatrix

$$W = \begin{bmatrix} 4 & 1 \\ 0 & 2 \end{bmatrix},$$

mit der eine minimale Laufzeit von 4h erreicht wird. Übersetzt heißt das, dass Maschine A vier Mal T1 erledigen und Maschine B ein Mal T1 und zwei Mal T2 übernehmen soll, damit die Laufzeit minimal ist.

Ich habe einen Algorithmus, der T und w in das entsprechende lineare Programm umwandelt und dieses anschließend löst in Python implementiert, sodass auch größere Task-Scheduling-Aufgaben mit beliebig aufteilbaren Tasks damit einfach gelöst werden können.

8.10 Dualität

Gegeben sei ein beliebiges lineares Programm der Form (in die man es immer umschreiben kann)

$$\max_x \{c^T x : Ax \leq b, x \geq 0\}$$

mit $A \in \mathbb{R}^{m \times n}$, $x, c \in \mathbb{R}^n$, $b \in \mathbb{R}^m$. Dann existiert zu diesem (primalen) LP immer ein sogenanntes duales LP. Dieses duale lineare Programm hat nicht das Ziel, das primale LP zu lösen (könnte man einfach mit dem Simplex-Algorithmus machen), sondern, den Optimalwert von $c^T x$ abzuschätzen indem man eine obere Schranke dafür findet. Die Lösung des dualen LP soll die kleinste mögliche solche Schranke sein. Doch wie kann man eine solche Schranke konstruieren?

8.10.1 Konstruktion des dualen LP und schwache Dualität

Dafür führen wir als Erstes einen Vektor $y \geq 0, y \in \mathbb{R}^m$ ein. Es gilt wegen der Nichtnegativität von y und der Nebenbedingung $Ax \leq b$ des primalen LP

$$y^T Ax \leq y^T b.$$

Das kann einfach komponentenweise überprüft werden. Die einzelnen Einträge von y können als 'Gewichtungen' für die in $Ax \leq b$ festgehaltenen Nebenbedingungen angesehen werden. Nehmen wir jetzt außerdem an, dass $A^T y \geq c$. Dann folgt aus der Nichtnegativität von x , dass

$$c^T x \leq y^T Ax.$$

In Kombination mit der vorherigen Ungleichung sieht man, dass

$$c^T x \leq y^T Ax \leq y^T b$$

und $y^T b$ somit eine von x unabhängige Schranke für den Optimalwert ist. Das heißt also, dass für alle zulässigen $y \geq 0$ (d.h. mit $A^T y \geq c$) $y^T b$ eine obere Schranke für den Optimalwert von $c^T x$ ist.

Das Ziel des dualen linearen Programms ist es, die kleinste mögliche Schranke für das primale LP zu finden, es lautet also

$$\min_y \{y^T b : A^T y \geq c, y \geq 0\}.$$

Weil immer $c^T x \leq y^T b$, ist auch

$$\max_x \{c^T x : Ax \leq b, x \geq 0\} \leq \min_y \{y^T b : A^T y \geq c, y \geq 0\}.$$

Das bezeichnet man als schwaches Prinzip der Dualität. Aus dem schwachen Dualitätsprinzip folgt, dass falls $c^T x^* = y^{*T} b$, dann x^* und y^* jeweils die Optima der beiden linearen Programme sind. Gäbe es nämlich ein x das größer oder ein y das kleiner als der gemeinsame Optimalwert ist, dann würde das den schwachen Dualitätssatz verletzen. Dieses x bzw. y kann es also nicht geben und x^* und y^* müssen optimal sein. Zusammengefasst: Aus der schwachen Dualität, die sich aus der Konstruktion des dualen LP ergibt folgt, dass

$$c^T x^* = y^{*T} b \implies x^* \text{ und } y^* \text{ sind optimal.}$$

8.10.2 Das duale LP des dualen LP ist das primale LP

Sei

$$\max_x \{c^T x : Ax \leq b, x \geq 0\}$$

das primale LP. Dann ist (wie vorhin gezeigt)

$$\min_y \{y^T b : A^T y \geq c, y \geq 0\}$$

das zugehörige duale Programm. Hier möchte ich zeigen, dass das umgekehrt auch der Fall ist. Das heißt, wir bilden zu dem dualen LP das duale LP und erkennen, dass dabei das ursprüngliche LP herauskommt:

$$\min_y \{y^T b : A^T y \geq c, y \geq 0\}$$

kann äquivalent umgeformt werden zu

$$= -\max_y \{(-b)^T y : (-A^T)y \leq -c, y \geq 0\}.$$

Das duale LP dazu ist nach der selben Logik wie beim Umformen von primal zu dual

$$-\min_x \{(-c)^T x : (-A^T)^T x \geq -b, x \geq 0\}.$$

Durch Umformen erhält man wieder

$$= \max_x \{c^T x : Ax \leq b, x \geq 0\},$$

was das primale LP ist. Das duale LP des dualen LP ist somit immer das primale LP.

8.10.3 Regeln für die Konstruktion des dualen LPs

- Wie vorhin schon gezeigt ist das duale LP eines Maximierungsproblems immer ein Minimierungsproblem und umgekehrt.
- b und c werden getauscht.
- A wird zu A^T
- Enthält die i -te (Un-)Gleichung ein $=/\geq/\leq$, dann ist $y_i \in \mathbb{R}/y_i \leq 0/y_i \geq 0$. Da das duale LP des dualen LP wieder das primale LP ist, muss diese Beziehung auch umgekehrt gelten.

8.10.4 Starke Dualität

Der starke Dualitätssatz besagt, dass sofern das primale LP beschränkt ist und mindestens eine zulässige Lösung existiert, das duale LP ebenfalls eine Lösung besitzt und immer

$$\max_x \{c^T x : Ax \leq b, x \geq 0\} = \min_y \{y^T b : A^T y \geq c, y \geq 0\}.$$

Das bedeutet, dass bei starker Dualität

$$x^* \text{ und } y^* \text{ sind optimal} \implies c^T x^* = y^{*T} b.$$

Falls kein Maximum des primalen LP wegen Unbeschränktheit existiert, dann existiert auch keine kleinste obere Schranke (unbeschränkt!) und das duale LP hat keine Lösung.

Die starke Dualität kann man folgendermaßen herleiten: ??